

Verifying wait-freedom for concurrent higher-order programs

Anonymous author

Anonymous affiliation

Anonymous author

Anonymous affiliation

Anonymous author

Anonymous affiliation

Abstract

Wait-freedom is the strongest non-blocking progress guarantee for concurrent data structures, ensuring that every operation completes in a finite number of steps regardless of interference from other threads. While verification of wait-freedom has been studied for first-order languages, verifying it for higher-order programming languages with general references remains an open challenge. In such languages, operations may be used by arbitrary, unverified higher-order clients, making it unclear how to even define wait-freedom formally in terms of programs' semantics, let alone prove it.

In this paper, we present the first framework for verifying wait-freedom of concurrent programs written in a higher-order language with general references. Our approach is based on the Lawyer concurrent separation logic which has been recently introduced for termination verification. We identify a specification pattern in the Lawyer logic that captures wait-freedom. To establish this connection formally, we extend Lawyer with a novel adequacy theorem that proves that programs which are proven correct in the Lawyer logic against a specification in this aforementioned specification pattern are wait-free. Proving wait-freedom requires us to show that all calls made to operations by any arbitrary client terminate. Thus, as part of formally proving wait-freedom, *i.e.*, as part of the proof of the adequacy theorem above, we need to prove that the behavior of the client of the data structure is safe in the sense that it does not break the internal invariants of the data structure, *e.g.*, by directly manipulating the data structure's internal state. To this end, we develop a logical relations model that establishes safety for all clients once and for all.

We demonstrate the effectiveness of our approach by proving wait-freedom for several representative examples, including higher-order functions such as the list map function, and a memory-efficient single-producer, single-consumer queue. For the latter, wait-freedom is conditional in that as the name suggests there can be at most one enqueueer thread and one dequeuer thread. To capture this formally we introduce the notion of restricted wait-freedom as a variant of wait-freedom that restricts the number of concurrent threads, and show how our approach can support reasoning about restricted wait-freedom. All our results have been mechanized on top of the Rocq Prover and using the Iris separation logic framework that Lawyer is also based on.

2012 ACM Subject Classification Software and its engineering → Formal software verification; Software and its engineering → Interactive theorem proving; Software and its engineering → Concurrency; Theory of computation → Logic and verification

Keywords and phrases separation logic, higher-order logic, concurrency, formal verification

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Any implementation of a concurrent data structure must account for possible interference between multiple operations on that data structure running concurrently. Done wrongly, it might lead to incorrect behavior of operations, or even worse, to undefined behavior of an entire program. The simplest way to ensure concurrent operations' correctness is to resort



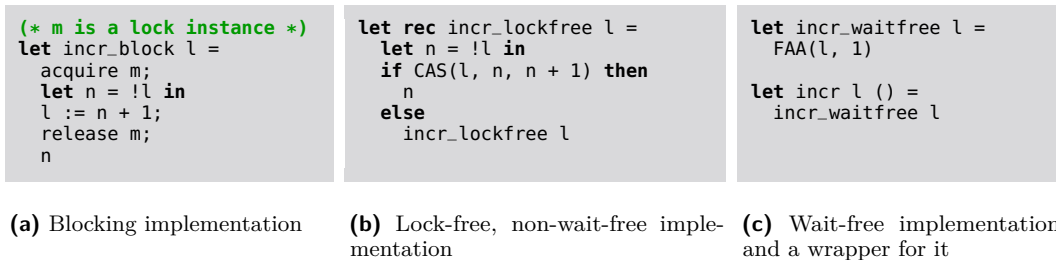
© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

XX:2 Verifying wait-freedom for concurrent higher-order programs



■ **Figure 1** Comparison of counter increment implementations; we use an OCaml-like syntax. Location l stores an integer

to *blocking concurrency*, *e.g.*, using a lock, allowing only one operation to progress at any given moment, and forcing others to wait for its completion. However, in many settings, *e.g.*, in operating system kernels [6], blocking concurrency is discouraged as it might lead to poor performance, and due to risk of deadlocking the entire system. Therefore, in such cases a *lock-free* implementation is more desirable. Lock-free data structure implementations guarantee that among the concurrently running operations, at least one of them must run to completion. Yet, lock-freedom does not guarantee that there is no starvation. That is, a specific call to a lock-free data structure's operation might not terminate because it is starved by other calls. To avoid starvation, one must use so-called *wait-free* [18] implementations. Wait-freedom is the strongest form of non-blocking concurrency. It guarantees that *every* individual call to the data structure operations must run to completion regardless of how many other operations of the same data structure run concurrently with it.

We illustrate the differences between blocking, lock-free, and wait-free concurrency using and simple and contrived example: a concurrent counter implementation featuring an increment operation, and a read operation (which we elide). Figure 1 shows multiple possible implementations of the increment operation. The implementation in Figure 1a allows only one thread at a time to access the counter, forcing all others to wait until the lock is released. The lock-free implementation (Figure 1b), implemented via an atomic Compare-And-Set (CAS) operation in a loop, does not wait for other concurrent operations on the counter to complete. It attempts to increment using the CAS operation, and if that fails, it retries. Note that if the CAS operation fails, that means that the value of the counter (stored in memory location l) must have changed since it was read on the line above the CAS operation. This in turn means that another increment operation must have succeeded in another thread. Thus, this implementation is lock-free. However, this lock-free implementation can starve threads, even under some fair schedulers. (Even though unlikely, a pathological but fair scheduler could schedule the thread, and subsequently preempt it right before the CAS operation, only to return to it, every time, after another increment has succeeded.) Finally, we have the wait-free implementation in Figure 1c. It uses the atomic Fetch-And-Add (FAA) operation which atomically increments the counter. Thus, the increment always finishes in a single, atomic step. In our discussions throughout the rest of the paper we will refer to this wait-free implementation of increment. To this end, we introduce a wrapper function `incr`, so that we can treat `incr l` as a function (of type `unit to unit`).

In this contrived example of a simple concurrent counter (Figure 1c) we essentially relegate the problem of making the data structure wait-free to the hardware using the FAA operation. In reality, as we will see below, for data structures ever so slightly more complex than this wait-free implementations are far from trivial to implement and to (formally) reason about.

<pre> let simple_op () = (* Could be allocated at address 0x42. *) let b = ref true in let rec loop () = if !b then () else loop () (* Should terminate immediately. *) in loop () </pre>	<pre> (* Parallel composition is written as *) let good_client = let r := ref simple_op in simple_op () !r () (* Direct memory access is written as [.] *) let bad_client = [0x42] := false simple_op () </pre>
---	---

■ **Figure 2** An example of a wait-free operation together with two possible clients

Indeed, such implementations must be designed to execute concurrently in a way that neither do they have to wait for the completion of other operations, nor are they impeded in a way by the other operations that forces them to retry. Therefore, a wait-free implementation of a data structure is often much more complicated than a lock-free or blocking implementation of the same data structure (see *e.g.* wait-free queue of Kogan and Petrank [27] versus the lock-free queue of Michael and Scott [32]). Thus, it is often also more challenging to verify.

In this paper, we show how to use a higher-order concurrent program logic to verify wait-freedom for programs written in a concurrent *higher-order* programming language with general references. Such languages achieve greater modularity by supporting features such as closures and first-class modules. Our work goes significantly beyond the state of the art [10, 30] which has only considered wait-freedom of programs in first-order programming languages without general references. Working with higher-order programs, and with general references, introduces many new challenges that one needs to answer. In fact, it is not even clear a priori how to formally define wait-freedom in our setting. Informally, it is often explained that a wait-free operation should be able to make progress in any program context under any scheduler. However, it is not entirely clear here what one means by “any program context”, in particular in our higher-order setting. A “context” (or *client*) of an operation can be any program that calls that operation, either directly or indirectly (through a reference), possibly concurrently—see *e.g.*, the `good_client` in Figure 2. However, it does not make sense to consider literally any client. In particular, the client’s behavior should be ‘safe’ in that it should not be able to violate data encapsulation of the wait-free data structure, *i.e.*, it should not be able to directly accessing data structure’s internal state. For a simple example of what can go wrong see the `bad_client` in Figure 2 which directly writes to `0x42` while concurrently running `simple_op`. The example `bad_client` could potentially violate an invariant of `simple_op` (namely the invariant that the local state variable `b` is always true) and thus break the wait-freedom of `simple_op`.

The example `bad_client` in Figure 2 shows that the progress guarantee of a wait-free operation crucially relies on its client being safe. Indeed, prior works on wait-freedom verification for first-order programs enforce safety of clients one way or another, but their approaches do not apply in our setting. One way to enforce clients’ safety is by using a programming language that syntactically separates accesses to the operation’s and client’s states [30]. This approach, however, is not viable for the higher-order language we consider. (In general, the introduction of higher-order store enables behaviors not present in a first-order setting, including turning a terminating program into non-terminating, as demonstrated by [11]). Another approach to ensuring safety of clients is by requiring them to be verified [10, 34]. In addition to not being applicable in our setting (explained below), this approach also inherently limits the applicability of the wait-freedom result: now every single client of potential interest must be verified in the corresponding framework. Rather, we take an approach that does not require one to verify individual clients but instead establishes

operations' progress within arbitrary clients. This also includes those clients out of reach of existing solutions, *e.g.*, higher-order clients.

In this work, we present the first solution to the verification of wait-freedom for a realistic, higher-order language. Our solution is based on the recent Lawyer logic [33], an extension of the Trillium [37] and Iris [26] frameworks. So far, Lawyer has only been used for establishing (fair) termination of closed programs. The key insight of our work is that a specific form of specifications given to a function forces that the function to terminate *in any context, without either blocking or starving*. That is, such Lawyer specifications essentially, for all intents and purposes codify wait-freedom in the Lawyer program logic. However, there are two main technical issues that prevent the Lawyer logic from directly being applicable to proving wait-freedom. Firstly, the so-called adequacy theorem of Lawyer is not applicable. The adequacy applies to a closed program, *i.e.*, not a single data structure, but rather the data structure and its client. Moreover, the adequacy theorem states that when a program is proven correct in Lawyer, then all of its threads must terminate, which does not consider non-terminating clients. Hence, in this work we state and prove a separate and novel adequacy theorem specifically for wait-freedom. Secondly, the Lawyer logic allows only composition of two programs as long as they are both verified in the Lawyer logic. Thus, if we were to only restrict ourselves to verified clients, we would also be restricted to only considering terminating clients. On the other hand, as we discussed above wait-freedom only makes sense when the client respects state encapsulation. Therefore, in our approach to verification of wait-freedom we prove once and for all that *all programs in our programming language respect state encapsulation*. For this purpose we construct a so-called logical relations model for our programming language. This is similar to how prior works on proving robust safety of programs [36, 14] have used logical relations models.

We use the approach outlined above for proving wait-freedom to prove wait-freedom of a number of example programs written in our higher-order language, including those that highlight these higher-order programming features. In particular, we modularly verify wait-freedom of the higher order map function for lists. That is, we show that `list_map f` is wait-free whenever it is applied to any wait-free argument `f`. We also show that our approach supports practical wait-free algorithms by verifying a memory-efficient single-producer, single-consumer queue [22]. An important point about this example, and in fact many practical wait-free algorithms, is that these algorithms in fact assume fixed upper bounds on the number of concurrent operations. Verification of this class of wait-free algorithms has been largely ignored by the prior work on verification of wait-freedom. Supporting reasoning about such examples formally requires us to introduce a notion of what we call *restricted* wait-freedom, and to slightly adjust our approach to verification of wait-freedom.

In summary, we make the following contributions in this paper:

- A formal definition of wait-freedom for higher-order languages together with two variations needed to account for realistic examples
- A Lawyer-based specification that internalizes the wait-freedom in the Lawyer logic
- A solution for verifying reasoning about arbitrary client programs based on combining a logical relations model for safety together with Lawyer logic's approach to liveness
- Verification of wait-freedom of a number of case studies including those not supported by prior work
- All results have been mechanized in the Rocq Prover (using classical axioms).

The rest of the paper is organized as follows. We formally define wait-freedom in Section 2 where we also state the adequacy theorem that reduces proving wait-freedom of an operation

to proving its wait-freedom specification in Lawyer. Then, in Section 3 we provide the necessary background on Iris. We then illustrate our approach in Section 4 by proving the wait-freedom specification for a simple example program. In Section 5 we continue by verifying more complicated programs which require adapting both the definition of wait-freedom and the Lawyer specification as we explained earlier. We explain the key ideas for proving our adequacy theorem in Section 6. Finally, we discuss a limitations of our solution in Section 7, compare it with related work in Section 8, and conclude in Section 9.

2 Overview

In this section, we formally define the notion of wait-freedom at the level of execution traces of the programming language under consideration. To avoid tedious direct reasoning about execution traces, our approach reduces proving this property to proving a specification with Lawyer program logic. We therefore briefly show the specification format of Lawyer and state the adequacy theorem for wait-freedom that establishes the aforementioned reduction.

Throughout the paper, we use the notation $m[k]$ to denote a key k lookup in a map-like structure m . That includes m being a list (and k an index in it), or a partial function $m : A \xrightarrow{\text{fin}} B$ for some types A and B (and k an element of type A). Moreover, we use this notation for the trace state lookup (see below). When we write $m[k]$, we implicitly assume that k belongs to the domain of m and thus the lookup is well-defined. Then, by $m[k \mapsto v]$ we denote the structure m with the key k updated to the value v .

Furthermore, we formally define traces and introduce a number of notations for them. For an arbitrary labeled transition system, a *trace* is a non-empty, possibly infinite alternating sequence of states and labels of that transition system that starts and ends with a state. We write $tr[i]$ and $tr\langle i \rangle$ to denote correspondingly the i th state and label of a trace tr . Then, $\text{dom}(tr)$ denotes the set of indices i for which $tr[i]$ is defined, *i.e.*, natural numbers between 0 and the (potentially infinite) length of tr .

2.1 Definition of wait-freedom for our language

We consider the programs written in a higher-order ML-like language $\lambda_{\text{ref}}^{\text{conc}}$. An excerpt of its semantics shown in Figure 3, for the full semantics see our Rocq development. Among the set Expr of $\lambda_{\text{ref}}^{\text{conc}}$ expressions, there are primitives for working with heap (*e.g.* $!e$ and $\text{free } e$) and for concurrency ($\text{fork}(e)$). A less common feature of $\lambda_{\text{ref}}^{\text{conc}}$ — to_int primitive — is explained further in Section 5.2. The heap is modeled as a partial map from locations to language values (which might include closures). In turn, the thread pool is modeled as a partial map from thread identifiers to expressions; the per-thread reductions are then naturally lifted to configurations (thread pool together with heap). The structure of evaluation contexts $ECtx$ establishes right-to-left call-by-value evaluation order. The $\text{fill}(E : ECtx)(e : \text{Expr})$ operator fills the hole $\bullet$ in the context E with the expression e ; we avoid the more common $E[e]$ notation, as in this paper it is used for lookups (see above).

Note that $\lambda_{\text{ref}}^{\text{conc}}$ forms a labeled transition system where states are configurations, labels are thread identifiers, and transitions are configuration reductions. In particular, for an execution trace etr of $\lambda_{\text{ref}}^{\text{conc}}$ and any index $i \in \text{dom } tr$, $tr[i]$ is the configuration before the i th step, and $etr\langle i \rangle$ is the identifier of the thread taking the i th step.

We define wait-freedom as a property of an *operation* $\text{op} : \text{Val}$ — an arbitrary one-argument function. Intuitively, op is wait-free if any “call” to op is followed by a “return” later in the execution. A “call” in $\lambda_{\text{ref}}^{\text{conc}}$ is simply an application $\text{op } a$ for some $(a : \text{Val})$. To identify a call in a thread pool $\mathcal{E}$, we need to provide a thread identifier $\tau : \text{Thread}$ and an evaluation

XX:6 Verifying wait-freedom for concurrent higher-order programs

Syntax

$x, y, f \in \text{Var}$	$l \in \text{Loc}$	$n \in \mathbb{Z}$	$b \in \mathbb{B}$
	$\odot ::= + \mid - \mid * \mid = \mid < \mid \dots$		
Val	$v ::= () \mid b \mid n \mid l \mid (v, v) \mid \text{inl } v \mid \text{inr } v \mid \text{rec } f(x) := e$		
Expr	$e ::= x \mid v \mid e \odot e \mid \text{ref}(e) \mid \text{if } e \text{ then } e \text{ else } e \mid \text{fork}(e) \mid !e \mid ee$ $\mid \text{to_int } e \mid \text{free } e \mid \text{fst } e \mid \text{snd } e$ $\mid \text{match } e \text{ with inl } x \Rightarrow e \mid \text{inr } y \Rightarrow e \text{ end} \mid \dots$		
ECtx	$E ::= \bullet \mid e \odot E \mid E \odot v \mid (e, E) \mid (E, v) \mid eE \mid Ev \mid \dots$		
Heap	$h \in \text{Loc} \xrightarrow{\text{fin}} \text{Val}$		
ThreadPool	$\mathcal{E} \in \text{Thread} \xrightarrow{\text{fin}} \text{Expr}$		
Config	$c ::= (\mathcal{E}, h)$		

Pure reductions

$$\begin{aligned}
 (\text{rec } f(x) := e) v &\xrightarrow{\text{pure}} e[\text{rec } f(x) := e/f][v/x] & (e, h) &\rightsquigarrow (e', h) & \text{if } e &\xrightarrow{\text{pure}} e' \\
 \text{if true then } e_1 \text{ else } e_2 &\xrightarrow{\text{pure}} e_1 & (\text{ref}(v), h) &\rightsquigarrow (\ell, h[\ell \mapsto v]) & \text{if } \ell &\notin \text{dom}(h) \\
 &\vdots & &\vdots &
 \end{aligned}$$

Per-thread reductions

Configuration reductions

$$\frac{(e, h) \rightsquigarrow (e', h')}{(\mathcal{E}[\tau \mapsto \text{fill } E e], h) \xrightarrow{\tau} (\mathcal{E}[\tau \mapsto \text{fill } E e'], h')} \quad \frac{\tau' \notin \text{dom}(\mathcal{E}) \cup \{\tau\} \quad \mathcal{E}' \triangleq \mathcal{E}[\tau \mapsto \text{fill } E ()][\tau' \mapsto e]}{(\mathcal{E}[\tau \mapsto \text{fill } E (\text{fork}(e))], h) \xrightarrow{\tau'} (\mathcal{E}', h)}$$

■ **Figure 3** An excerpt of $\lambda_{\text{ref}}^{\text{conc}}$ semantics

context $E : \text{ECtx}$. Then, a return in our approach is simply a value, similarly identified with a thread identifier and an evaluation context.

► **Definition 1** (Call and return in a thread pool).

$$\text{Call}(\mathcal{E}, \tau, E, \text{op}) \triangleq \exists a \in \text{Val}. \mathcal{E}[\tau] = \text{fill } E(\text{op } a) \quad \text{Ret}(\mathcal{E}, \tau, E) \triangleq \exists r \in \text{Val}. \mathcal{E}[\tau] = \text{fill } E r$$

Requiring that every call to op is followed by return only makes sense when the calling thread is kept being scheduled until the return. This is captured by the following definition.

► **Definition 2** (Eventual return of calls). *For an operation $\text{op} : \text{Val}$, execution trace etr and thread identifier τ , we say that all τ 's calls to op in etr eventually return iff*

$$\begin{aligned}
 \text{alwaysReturnsInTrace}(\text{op}, \text{etr}, \tau) &\triangleq \forall (E : \text{ECtx}), i. \\
 \text{Call}(\text{etr}[i].1, \tau, E, \text{op}) &\Rightarrow \text{schedUntilRet}(\text{etr}, \tau, E, i) \Rightarrow \exists j \geq i. \text{Ret}(\text{etr}[j].1, \tau, E) \\
 \text{where } \text{schedUntilRet}(\text{etr}, \tau, E, i) &\triangleq \forall j \geq i. j \in \text{dom } \text{etr} \Rightarrow \\
 \neg(\exists k \in [i..j]. \text{Ret}(\text{etr}[k].1, \tau, E)) &\Rightarrow \exists p \geq j. \text{etr}[p] = \tau,
 \end{aligned}$$

Above, $\text{schedUntilRet}(\text{etr}, \tau, E, i)$ says that, if the thread τ has called (in terms of Definition 1) op at index i and has not returned from it yet, then τ is guaranteed to be eventually scheduled. Note that each call requires a separate return, matched by an evaluation context. Indeed, in $\lambda_{\text{ref}}^{\text{conc}}$, an evaluation context is preserved by configuration reductions until the expression under that context reduces to a value. Therefore, the contexts of, e.g., nested calls will be different (with every subsequent call occurring in a "deeper" context).

Finally, we consider wait-freedom in executions with the following starting configurations:

- 231 ■ The program of every thread must be a substitution of the wait-free operation's code
232 into some expression with a free variable x . Moreover, this expression must not contain
233 hard-coded locations and pointer arithmetic (denoted by $noLocs$): the example from
234 Figure 2 shows why it is necessary to restrict random memory access in clients.
- 235 ■ The data structure accessed by the operation must be initialized. What exactly “initial-
236 ization” means depends on the operation (see an example below), therefore our definition
237 of wait-freedom is parameterized with the set of initialized configurations.

238 As a result, we define wait-freedom of an operation as requiring that operation to always
239 return in any thread of any execution that starts in an appropriately initialized configuration.

240 ► **Definition 3** (Wait-freedom). *Given a set of initialized configurations $C \subseteq Config$, we
241 define wait-freedom of an operation op as*

$$242 \quad waitFree_C(op) \triangleq \forall etr. \bigwedge_{\tau \mapsto e \in etr[0].1} (\exists e'. e = e'[op/x] \wedge noLocs(e')) \Rightarrow$$

$$243 \quad etr[0] \in C \Rightarrow \forall \tau. alwaysReturnsInTrace(op, etr, \tau)$$

244 For example, in Section 4 we show that a FAA-based `incr` operation of counter (Figure 1c)
245 is wait-free if the counter location l is initialized with an integer. In terms of Definition 3, it
246 is denoted as $waitFree_{hasInt(l)}(incr\ l)$, where $hasInt(l) \triangleq \{c \mid c.2[l] \in \mathbb{Z}\}$.

247 We note that, while Definition 3 captures the behavior of simple wait-free operations,
248 more complicated classes of algorithms only satisfy weaker forms of that property. We will
249 see examples of it in Section 5.1 and Section 5.2.

250 2.2 Proving wait-freedom with Lawyer logic

251 The notion of wait-freedom as defined in Definition 3 is formulated at the level of execution
252 traces, which are notoriously hard to reason about directly. To avoid that, our approach
253 reduces establishing wait-freedom of a given operation to verifying this operation with
254 Lawyer[33] — a concurrent separation logic built on top of Iris [26] and Trillium [37] frameworks.
255 We explain the details of Lawyer logic in the subsequent sections; here, we present the high-
256 level form of the specification that a wait-free operation must satisfy.

257 ► **Definition 4** (Lawyer specification of wait-freedom). *For a set of initialized configurations
258 $C \subseteq Config$, we define the Lawyer specification of op 's wait-freedom as*

$$259 \quad WaitFreeSpec_C(op) \triangleq \forall c \in C. heap_repr(c) \Rightarrow NoInfExec(op) \wedge PresInv(op)$$

260 The above specification can be read as follows. For any initialized configuration c , we
261 assume the ownership of c 's entire heap, represented as an Iris proposition $heap_repr(c)$.
262 Given that (reading $\Rightarrow$ as an implication), we prove two specifications for the operation op .
263 First, $NoInfExec(op)$ states that for any argument $a : Val$, execution of $op\ a$ cannot be infinite.
264 Second, $PresInv(op)$ states that for any a , execution of $op\ a$ preserves the internal invariants
265 that op relies on (*e.g.* that the location l of counter in Figure 1c always stores a number).

266 Our adequacy theorem states that the specification above indeed implies wait-freedom.

267 ► **Theorem 5** (Adequacy). *If $WaitFreeSpec_C(op)$ is provable in Lawyer, then $waitFree_C(op)$
268 holds in the meta-logic.*

269 As we note in Section 2.1, the notion of wait-freedom for more complicated classes of
270 operations is more elaborate. For these, we use slightly different versions of Definition 4 and
271 Theorem 5 described in Section 5.

$$P, Q ::= \text{True} \mid \text{False} \mid P \wedge Q \mid P \vee Q \mid P \Rightarrow Q \mid \forall x. P \mid \exists x. P \mid P * Q \mid \\ \{P\} (e : \text{Expr}) \{v.Q\} \mid (\ell : \text{Loc}) \mapsto (v : \text{Val}) \mid \boxed{P} \mid \text{toks}_N(k) \mid P \Rightarrow Q \mid \dots$$

■ **Figure 4** Propositions of Iris logic. Here, *Expr*, *Loc* and *Val* refer to those from Figure 3

3 Background: Iris logic

The Lawyer logic [33] that we use to verify wait-freedom is built on top of Trillium framework [37], which is in turn an extension of the Iris framework [26]. In this section, we provide the general background on Iris by proving two specifications (not relevant to wait-freedom) for the counter increment operation `incr` from Figure 1c. Later in Section 4, we actually verify the wait-freedom specification for `incr` with Lawyer logic, concentrating on how it differs from Iris. We only address Trillium framework specifically in Theorem 10.

Strictly speaking, Iris is a *framework* that only provides the minimal base logic and allows to define custom program logics on top of it. We consider the specific instantiation of Iris shown in Figure 4 (inspired by one used in Iris Lecture Notes [4]); we explain the constructs of Figure 4 as we go. In this section, by “Iris” we mean that specific instantiation. Similarly, there are many possible specifications for `incr` even within that Iris instantiation; we consider those that illustrate the concepts used in this paper.

The simplest application of Iris is using it as a *sequential Hoare logic*. Such logics reduce verifying properties of sequential programs to proving *Hoare triples* for them. Such a triple is written as $\{P\} e \{v.Q\}$ and intuitively means that if the precondition P holds when an execution of program e starts, then the execution does not get stuck, and that whenever e reduces to some value v , the postcondition $Q(v)$ holds. The program state described by pre- and post-conditions can include, among others, the *points-to* propositions. Such a proposition is written as $\ell \mapsto v$, meaning that the heap location ℓ currently has value v ; moreover, it asserts exclusive ownership of ℓ — the notion we address later. Finally, Iris includes the standard primitives of first-order logic such as propositional connectives and quantifiers.

With the intuition above, we can specify the program in Figure 1c as follows:

$$\forall \ell : \text{Loc}, n : \mathbb{Z}. \{\ell \mapsto n\} \text{incr } \ell () \{m. m = n \wedge \ell \mapsto (n + 1)\} \quad (\text{incr-spec-seq})$$

This specification states that regardless of the location and the current value of counter, the increment operation executes without getting stuck, that its return value equals to the current counter value, and that upon return this value is actually incremented. Indeed, the adequacy theorem of Iris [24] allows to establish this property.

Proving simple triples such as `incr-spec-seq` amounts to applying primitive proof rules on every execution step. Figure 5 shows a subset of Iris proof rules required to verify the specifications in this section. In particular, as execution of `incr` $\ell ()$ consists of two beta-reductions and one FAA step, we need the rules IRIS-BETA and IRIS-FAA. The former states that taking a beta-reduction does not change the program state, so the reduct can be verified with the same pre- and post-condition. The latter states that fetch-and-add updates the value stored under the corresponding location and returns the previously stored value.

To prove `incr-spec-seq`, we first introduce the universally quantified ℓ and n . Then, we apply IRIS-BETA twice to reduce `incr` and `incr_waitfree`. Afterwards, we apply IRIS-FAA to prove $\{\ell \mapsto n\} \text{FAA}(\ell, 1) \{m. m = n \wedge \ell \mapsto (n + 1)\}$ and finish the proof.

However, the specification above and the fragment of Iris logic described so far have

$$\begin{array}{c}
\text{IRIS-BETA} \\
\frac{\{P\} e[v/x] \{Q\}}{\{P\} (\lambda x. e) v \{Q\}}
\end{array}
\quad
\begin{array}{c}
\text{IRIS-FAA} \\
\frac{}{\{\ell \mapsto n\} \mathbf{FAA}(\ell, k) \{m. m = n \wedge \ell \mapsto (n + k)\}}
\end{array}
\quad
\begin{array}{c}
\text{IRIS-HT-FRAME} \\
\frac{\{P\} e \{Q\}}{\{P * R\} e \{Q * R\}}
\end{array}$$

$$\begin{array}{c}
\text{IRIS-HT-CSQ} \\
\frac{\{P'\} e \{Q'\} \quad P \Rightarrow P' \quad \forall v. Q'(v) \Rightarrow Q(v)}{\{P\} e \{Q\}}
\end{array}
\quad
\begin{array}{c}
\text{IRIS-INV-ACC} \\
\frac{\{P * Q\} e \{P * R\} \quad e \text{ is atomic}}{\{[P] * Q\} e \{R\}}
\end{array}$$

■ **Figure 5** Selected rules of Iris program logic

a number of disadvantages. First, the pre- and postconditions of Hoare triples describe the entire program state, whereas the rules such as IRIS-FAA are intended to operate only with a part of it. Then, the specification `incr-spec-seq` is *sequential*: it assumes that the thread executing `incr ℓ` always knows the exact value under `ℓ`. This is not a realistic case for *concurrent* libraries: other threads can interfere by running `incr ℓ` on their own.

Below, we illustrate the use of Iris as a *concurrent separation logic* by proving a more elaborate specification for `incr`. There are many possible concurrent specifications for this operation; Iris Lecture Notes [4] provide a number of them. We choose another, *token-based* specification because the notion of “tokens” is used a few times throughout the paper: it resembles the Lawyer’s notion of fuel (Section 4), and is used to specify restricted wait-freedom (Section 5.2). As said above, in the concurrent setting it is impossible to always hold the exact knowledge of the current counter value. However, by limiting the number of possible increments, one can approximate the current counter value when reading it. Namely, when instantiating the counter, the user chooses its maximal possible value N . Upon the instantiation, the user receives N non-replenishable *tokens*. A token is spent on every counter increment. Unused tokens can be presented upon reading the counter; the more tokens are presented, the smaller the counter value must be. Formally, we can specify the counter operations as follows (where `mk_cnt ()` $\triangleq$ `ref 0` and `read ℓ` $\triangleq$ `!ℓ`):

$$\begin{array}{ll}
\forall N : \mathbb{N}. & \{\text{True}\} \text{mk_cnt} () \left\{ \ell. [\text{cntInvUpto}(\ell, N)] * \text{toks}_N(N) \right\} \\
\forall \ell, N. & \left\{ [\text{cntInvUpto}(\ell, N)] * \text{toks}_N(1) \right\} \text{incr } \ell () \{m. 0 \leq m\} \\
\forall \ell, N, k. & \left\{ [\text{cntInvUpto}(\ell, N)] * \text{toks}_N(k) \right\} \text{read } \ell \{m. 0 \leq m \leq (N - k) * \text{toks}_N(k)\}
\end{array}$$

This specification features all the main concepts of Iris used to verify concurrent programs. First, it shows that Iris is a *separation logic* that treats propositions as *disjoint resources*. Resources represent both the state of the program (*e.g.* points-to propositions) and the auxiliary information (*e.g.* the number of tokens). The *separating conjunction* $P * Q$ asserts ownership of a resource that can be split into two disjoint parts, satisfying P and Q correspondingly. The notion of being disjoint depends on particular resource kind; for points-to, $l_1 \mapsto v_1 * l_2 \mapsto v_2$ implies $l_1 \neq l_2$, *i.e.*, ownership of a memory location is *exclusive*.

Then, $\text{toks}_N(k)$ is an example of a *ghost resource*, *i.e.* one that is not directly tied to the program state. The Iris user can choose a number of ghost resources to support the verification process. In Figure 4, we assume that Iris instantiation contains tokens resource $\text{toks}_N(k)$ that asserts ownership of k out of N tokens and satisfies the following laws:

$$\text{toks}_N(k) * \text{toks}_N(m) \dashv\vdash \text{toks}_N(k + m) \quad \text{toks}_N(k) \vdash k \leq N$$

XX:10 Verifying wait-freedom for concurrent higher-order programs

Finally, the specifications above feature an *invariant* $\boxed{\text{cntInvUpto}(\ell, N)}$. Once established, an invariant proposition $\boxed{P}$ means that P will hold throughout program's execution. More specifically, it is guaranteed to hold between the execution steps, so every thread can rely on P to justify its own step, but must reestablish P after taking that step. Importantly, invariants (but not their content in general) are duplicable. That is, $\boxed{P} \vdash \boxed{P} * \boxed{P}$; with that, exclusive resources such as points-to can be placed in the invariant and accessed by every thread through its own copy of invariant. For the specification above, we define the counter invariant as follows, capturing the intuition of spending a token for every increment:

$$\boxed{\text{cntInvUpto}(\ell, N)} \triangleq \exists k \geq 0. \ell \mapsto k * \text{toks}_N(k)$$

We verify `mk_cnt` and `read` in the supplementary material; below, we show how the verification of `incr` changes with the new specification. We verify the two beta-reductions as before by applying IRIS-BETA twice and end up with the following triple to prove:

$$\left\{ \boxed{\text{cntInvUpto}(\ell, N)} * \text{toks}_N(1) \right\} \text{FAA}(\ell, 1) \{m. 0 \leq m\}$$

To proceed, we need the points-to resource in the invariant. For this, we use the invariant access rule IRIS-INV-ACC that allows to open the invariant (*i.e.* obtain the underlying resource) for a single execution step and requires to close it back after that step¹. Applying this rule to the triple above and unfolding our invariant, we are left proving

$$\{(\exists k \geq 0. \ell \mapsto k * \text{toks}_N(k)) * \text{toks}_N(1)\} \text{FAA}(\ell, 1) \{m. (\exists k \geq 0. \ell \mapsto k * \text{toks}_N(k)) * 0 \leq m\}$$

Hoare triples enjoy the rule of consequence, *i.e.* IRIS-HT-CSQ. Using it along with the aforementioned law for combining tokens and standard rules for separation logic, we reduce the triple above to the following, for some $k \geq 0$:

$$\{\ell \mapsto k * \text{toks}_N(k+1)\} \text{FAA}(\ell, 1) \{m. m = k * \ell \mapsto (k+1) * \text{toks}_N(k+1)\}$$

339 To verify the FAA step, we first apply the frame rule IRIS-HT-FRAME that strips the irrelevant
340 resources from both the pre- and post-condition. We close the proof by applying IRIS-FAA to
341 verify the remaining triple $\{\ell \mapsto k\} \text{FAA}(\ell, 1) \{m. m = k * \ell \mapsto (k+1)\}$.

342 In the subsequent sections, the specifications will be verified similarly. There will be
343 slight differences in the Hoare triples format and the exact form of proof rules, and we will
344 use other ghost resources, but the overall idea will be the same: establish the invariant and
345 then verify every execution step with the corresponding proof rule.

4 Verifying wait-freedom in Lawyer logic

347 In this section, we show how an operation is verified against the wait-freedom specification
348 of Definition 4. As an example, we consider `incr l` from Figure 1c. Specifically, we verify
349 $\text{WaitFreeSpec}_{\text{hasInt}(l)}(\text{incr } l)$ in the Lawyer logic for any location l . Doing so allows us to
350 use Theorem 5 to conclude $\text{waitFree}_{\text{hasInt}(l)}(\text{incr } l)$ holds at the meta-level. Most of the
351 verification process amounts to proving the Hoare triples similarly to how it was done in
352 Section 3, although the details of program logics are different.

¹ To prevent unsoundly opening the invariant twice in the same step, Iris features *masks* of Hoare triples and *namespaces* of invariants. We omit them for simplicity. Moreover, in general the resource under invariant is obtained under the *later modality*; for this particular invariant it can be ignored.

353 Consider some counter location l . By definition of `WaitFreeSpec`, we need to prove

$$354 \quad \forall c \in \text{hasInt}(l). \text{heap_repr}(c) \Rightarrow \text{NoInfExec}(\text{incr } l) \wedge \text{PresInv}(\text{incr } l)$$

To do so, we take an arbitrary configuration $c : \text{Config}$ for which we have $c.2[l] = k$ for some $k \in \mathbb{Z}$, and further assume ownership of each heap location of c with separating conjunction over all available points-to resources:

$$\text{heap_repr}(c) \triangleq \bigstar_{\{(\ell, v) \mid c.2[\ell] = v\}} \ell \mapsto v$$

With the ownership of the entire heap, we will prove the specifications $\text{NoInfExec}(\text{incr } l)$ and $\text{PresInv}(\text{incr } l)$. Proving both relies on the fact that l always stores some number. For this, we use a counter module invariant (which is even simpler than one used in Section 3):

$$\text{cntInv} \triangleq \boxed{\exists k \in \mathbb{Z}. l \mapsto k}$$

355 Note that the definition of `WaitFreeSpec` features a so-called *viewshift*, written $\Rightarrow$. View-
 356 shifts are similar to logical implication, but additionally allow to access and establish the
 357 invariants so as to establish the consequent. Thus, given ownership of the entire heap, and
 358 the fact that l currently points to some integer, we establish cntInv by providing $l \mapsto k$
 359 (ignoring the rest of the heap). From that point on we assume that cntInv holds while proving
 360 the specifications, adding it to preconditions of Hoare triples under consideration.

The first specification we need to prove, $\text{NoInfExec}(\text{incr } l)$, informally captures that execution of our operation applied to any argument cannot run forever. Formally, it is defined as a dependent pair of a natural number and a specification that mentions it²:

$$\text{NoInfExec}(\text{op}) \triangleq \{\mathbf{F} : \mathbb{N} \mid \forall \tau : \text{Thread}, \pi : \text{Phase}, a : \text{Val}. \{\mathbf{F} \cdot \text{bar}_\circ \pi * \text{ph}_\circ \tau \pi\} \text{ op } a \{ _ . \text{ph}_\circ \tau \pi \}^\tau\}$$

361 The number $\mathbf{F}$ is (an over-approximation of) the upper bound on the execution time of
 362 op . The actual specification for op is defined as a *Lawyer Hoare triple*. Lawyer [33] is
 363 an instantiation of the Trillium framework [37], which is in turn an extension of Iris that
 364 establishes *refinement* of a program to a transition system. Thus, compared to the Iris triples
 365 introduced in Section 3, the Hoare triples in Lawyer additionally enforce that every single
 366 step of the program is matched by a transition in so-called Obligations Model—a transition
 367 system that Lawyer uses to represent terminating programs. Proving a Lawyer Hoare triple
 368 for a program implies that any execution trace of that program refines some trace of the
 369 Obligations Model. As all traces of the Obligation Model are finite, this refinement implies
 370 termination of verified programs. Note that the Lawyer triples are additionally annotated
 371 with the thread identifier, because the thread identity is important for proving refinement.

372 The precondition and postcondition of the triple in $\text{NoInfExec}(\text{op})$ consist of the ghost
 373 resources of Lawyer logic. The precondition can be read as an assignment of $\mathbf{F}$ barrels of *fuel*
 374 to the thread τ , with an indirection via the *phase* π that marks the barrels and is assigned
 375 to τ . The fuel is represented by the $\mathbf{F} \cdot \text{bar}_\circ \pi$ resource (a separating conjunction of $\mathbf{F}$ many
 376 individual barrels $\text{bar}_\circ \pi$). The phase assignment is written in the Lawyer logic as $\text{ph}_\circ \tau \pi$.
 377 The fuel resource represents the number of remaining steps the program is allowed to take:
 378 as we will see later, verifying every single execution step consumes one barrel of fuel, similar
 379 to how every counter increment consumed a token in Section 3. We remark that in Lawyer

² The reason why $\mathbf{F}$ cannot be existentially quantified inside the triple is related to the issues of step-indexing which are described in detail by Spies *et al.* [35].

XX:12 Verifying wait-freedom for concurrent higher-order programs

[33], both the fuel and the phase of resources have one extra parameter each. Technically, we assign these parameters the values that are largely irrelevant for this presentation; instead of showing these values, we just mark both resources with a $\circ$ subscript. In the appendix, we argue why this particular specification pattern of $\text{NoInfExec}(\text{op})$, as well as certain restrictions on the operation's invariants, captures the nature of wait-freedom in the Lawyer logic.

To prove $\text{NoInfExec}(\text{incr } l)$, we take $\mathbf{F}$ to be 3. Then, we prove, for any τ , π , and a :

$$\{\text{cntInv} * 3 \cdot \text{bar}_\circ \pi * \text{ph}_\circ \tau \pi\} \text{incr } l a \{ _ . \text{ph}_\circ \tau \pi \}^\tau$$

The triple above essentially states that we can spend up to 3 barrels of fuel to complete the execution of $\text{op } a$. To prove it, we use the rules³ of the Lawyer logic. Compared to the proof rules presented in Section 3, the main difference is that the rules of Lawyer feature Lawyer-specific resources besides the Iris ones. This is because Lawyer requires to match every step of execution with a transition in Obligations Model, which in turn requires resources such as fuel. Specifically, the Lawyer rules for beta-reductions and FAA are the following:

LAWYER-BETA-FUEL

$$\{\text{ph}_\circ \tau \pi * P\} e[v/x] \{\text{ph}_\circ \tau \pi * Q\}^\tau \vdash \{\text{bar}_\circ \pi * \text{ph}_\circ \tau \pi * P\} (\lambda x. e) v \{\text{ph}_\circ \tau \pi * Q\}^\tau$$

LAWYER-FAA-FUEL

$$\{\text{bar}_\circ \pi * \text{ph}_\circ \tau \pi * \ell \mapsto n\} \text{FAA}(\ell, k) \{m. m = n * \ell \mapsto (n + k) * \text{ph}_\circ \tau \pi\}^\tau$$

After applying LAWYER-BETA-FUEL twice, we are left to prove the following:

$$\{\text{cntInv} * \text{bar}_\circ \pi * \text{ph}_\circ \tau \pi\} \text{FAA}(l, 1) \{ _ . \text{ph}_\circ \tau \pi \}^\tau$$

Proving it requires opening the invariant, for which Lawyer inherits the IRIS-INV-ACC rule unchanged. Applying it, along with IRIS-HT-CSQ and LAWYER-FAA-FUEL , finishes the proof.

The next specification that we need to prove, *i.e.*, $\text{PresInv}(\text{incr } l)$, is defined as a yet another Hoare triple as follows:

$$\text{PresInv}(\text{op}) \triangleq \forall \tau : \text{Thread}, a : \text{Val}. \{\text{RS}_\mathbf{V}(a)\}^\sharp \text{op } a \{v. \text{RS}_\mathbf{V}(v)\}^\tau$$

As its name suggests, this triple guarantees that the execution of op preserves the established invariants. In fact, the same holds for Hoare triples of Iris and Lawyer; the crucial difference is that this triple allows the execution to get stuck (denoted by $\sharp$ superscript). This triple does not establish refinement to any model, but for technical reasons it is annotated with the thread identifier like a Lawyer triple⁴.

As we will explain in Section 6.2, such Hoare triples ensure that op can be used by an arbitrary client program, and that it would not violate the internal invariants of op . Technically, a client program *uses* op by providing it a *safe* argument, captured by the predicate $\text{RS}_\mathbf{V}(a)$, and expects op to return a safe value back. We will explain what it means to be safe and motivate why it is necessary to restrict only to safe values in Section 6.2.

Since $\text{PresInv}(\text{incr } l)$ does not establish refinement to a model, its proof follows the process described in Section 3. We ignore the $\text{RS}_\mathbf{V}(a)$ resource, because incr ignores the input argument a . The rules used to verify beta-reductions and FAA are exactly the same as in Section 3 (except for the different type of triple). Similarly to the proof above, we use cntInv to verify the FAA step. The main difference there is that the return value v is not ignored—it needs to be shown to be safe, *i.e.* $\text{RS}_\mathbf{V}(v)$. However, we know that FAA returns an integer, and, as we will see in Section 6.2, all integers are safe, *i.e.*, $\forall n : \mathbb{Z}. \vdash \text{RS}_\mathbf{V}(n)$.

³ The rules that we present below are derived from the more general rules of Lawyer logic.

⁴ Technically, this Hoare triple is defined by directly instantiating the Trillium's program logic with a trivial, always looping transition system.

```

let rec list_map f l =
  match l with
  | None => None
  | Some p => Some (f (fst p), list_map f (snd p))
end

```

$$\text{isList}(v : \text{Val}) \triangleq v = \text{None} \vee \exists h, v'. v = \text{Some}(h, v') \wedge \text{isList}(v')$$

(a) Implementation

(b) Representation of lists in $\lambda_{\text{ref}}^{\text{conc}}$

Figure 6 Higher-order list mapping function

5 Case studies

As we discussed in the Introduction, while the simple definition of wait-freedom that we used in Sections 2 and 4 works for many simple programs, for more involved programs, this definition needs to be adjusted. Specifically, Definition 3 follows the classical definition of wait-freedom [18]: it requires an operation to progress safely (*i.e.*, without getting stuck) regardless of its input arguments and the number of concurrent operations. However, each of these conditions can affect the safe progress in practice.

In this section, we consider two illustrative examples that motivate two independent extensions to our wait-freedom verification approach. First, we verify (higher-order) functions that potentially get stuck on “wrong” argument. Second, we verify a data structure that restricts the number of operations that can run concurrently at a given time. The latter is especially important, as many realistic wait-free data structures [15, 29, 38] rely on this restriction. We note that these changes account for different types of progress properties, not for each individual example; we believe that the case studies exhibiting the same property can be verified with the same specification and the adequacy theorem.

Throughout this section we will use the helper definitions $\text{None} \triangleq \text{inl}()$ and $\text{Some } v \triangleq \text{inr } v$ and similarly define a corresponding notation for in pattern matching where we simply write $\text{match } e \text{ with } \text{None} \Rightarrow e_1 \mid \text{Some } v \Rightarrow e_2 \text{ end}$.

5.1 Modular verification of higher-order, possibly-stuck operations

Consider the higher-order `list_map` function in Figure 6a. It applies the given function `f` to all elements of the list `l`. In our language, lists are values satisfying the recursive predicate `isList` defined in Figure 6b.

Note that if the input function `f` does not run forever for any argument, the partially applied `list_map f` does not run forever for any argument either. However, it might get stuck even if `f` never does: if its second argument `l` does not satisfy `isList`, then `list_map` will get stuck, *e.g.*, on taking projections. Thus, the behavior of calls to `list_map f` is captured by the following variant of Definition 2, with the changes highlighted:

► **Definition 6** (Eventual return or stuckness of calls to `op`). *For an operation $\text{op} : \text{Val}$, execution trace etr and thread identifier τ , we say that all τ ’s calls to op in etr eventually return or get stuck, written $\text{alwaysReturnsInTrace}^{\sharp}(\text{op}, \text{etr}, \tau)$, iff*

$$\begin{aligned} \text{alwaysReturnsInTrace}^{\sharp}(\text{op}, \text{etr}, \tau) &\triangleq \\ &\forall \mathcal{E}, i. \text{Call}(\text{etr}[i].1, \tau, E, \text{op}) \wedge \text{schedUntilRet}(\text{etr}, \tau, E, i) \Rightarrow \\ &\quad \exists j \geq i. \text{Ret}(\text{etr}[j].1, \tau, E) \vee \text{stuck}(\text{etr}[j], \tau) \\ &\text{where stuckness is defined as } \text{stuck}(c, \tau) \triangleq \neg \exists c'. c \xrightarrow{\tau} c' \end{aligned}$$

XX:14 Verifying wait-freedom for concurrent higher-order programs

Similarly, we define possibly-stuck wait-freedom $waitFree^{\zeta}$ as a variant of Definition 3 that uses $alwaysReturnsInTrace^{\zeta}$ instead of $alwaysReturnsInTrace$.

Our approach to proving wait-freedom supports verifying possibly-stuck wait-freedom with minimal changes compared to the ordinary wait-freedom discussed in earlier sections. That is, we prove a variant of Theorem 5 that establishes possibly-stuck wait-freedom of an operation, given that it satisfies the Lawyer specification $WaitFreeSpec^{\zeta}$, which, similarly to definitions above, is a weakening of $WaitFreeSpec$ that allows the operation to get stuck.

Now, we can apply the approach outlined in Sections 2 and 4 to prove that `list_map f` is possibly-stuck wait-free. Crucially, we do so modularly by verifying both `list_map` and `f` independently of one another. Formally, we prove the following higher-order possibly-stuck wait-freedom specification for `list_map`:

$$\forall f, C. WaitFreeSpec_C^{\zeta}(f) \Rightarrow WaitFreeSpec_C^{\zeta}(list_map\ f) \quad (\text{list-map-HO-spec})$$

As in Section 4, we prove the two specifications for `list_map f`. For each of them, we make use of the corresponding specification for `f`. The logical inference rules used in the process are the same used in Section 4, with the addition of rules to verify stuck computations, *e.g.*, the following Hoare triple for projecting of the unit value: $\{\text{True}\}^{\zeta} \text{fst}() \{P\}^{\tau}$.

Given the (list-map-HO-spec) spec above, together with the fact that ordinary Hoare triples imply possibly-stuck Hoare triples, we can reuse the already proven (possibly-stuck) wait-freedom specifications to derive possibly-stuck wait-freedom of `list_map` partially applied to specific function argument. For example, by weakening the Hoare triple that we proved in Section 4, we obtain a proof of $WaitFreeSpec_{hasInt(l)}^{\zeta}(list_map(\text{incr } l))$ for any location l , thus establishing $waitFree_{hasInt(l)}^{\zeta}(list_map(\text{incr } l))$.

Finally, we note that proving $WaitFreeSpec_C^{\zeta}(list_map\ f)$ requires one more generalization to the specification. The amount of fuel consumed by `list_map f` depends on the length of the input list; to account for that, we replace the fuel amount $\mathbf{F}$ in `NoInfExec` with a *fuel function*. In fact, the fuel amount used in all other examples is simply defined as a constant fuel function. We refer the reader to our Rocq mechanization for details.

5.2 Restricted wait-freedom

Many practical implementations of wait-free algorithms [22, 28, 38, 15] assume an extra condition not accounted for by Definition 3: that the number of concurrent operations has a fixed upper bound. An example of such implementation is shown in Figure 7a. This implementation of a concurrent wait-free queue has two uncommon features. First, besides adding and removing elements to the queue, it allows reading the head element without removing it. Second, it optimizes the memory usage by deallocating the head node when it is removed. Supporting both of these features, and simultaneously safety and Linearizability is highly non-trivial. Therefore, the implementation in Figure 7a limits the concurrency to just two threads: an enqueueer thread that can both enqueue elements and read the queue head, and a dequeuer thread that can both dequeue elements and read the queue head.

Verifying such algorithms, which we call *restricted wait-free*, requires multiple changes to our approach as we have described so far. First of all, we define two wrapper functions `enqueueer` and `dequeuer` (see Figure 7b) which wrap the possible operations of the queue for the enqueueer and dequeuer threads explained above into a single operation respectively—each function checks its argument and dispatches the appropriate queue operation accordingly. Then, for technical reasons that we explain in Section 7, we restrict the queue to only store integers. To that end, the `enqueueer` wrapper includes a call to the `to_int` primitive (which


```

(* Node = {val: Value; next: pointer to Node}; OldHeadVal: Value *)
(* Head, Tail, BeingRead, FreeLater : pointer to Node *)

(* methods used by "enqueueer" thread: *)
let read_head_e () =
  let cur_head = !Head in
  if (cur_head == !Tail) then None
  else
    BeingRead := cur_head;
    let v =
      if (cur_head == !Head) then
        cur_head.val
      else
        !OldHeadVal
    in Some v
let enqueue v =
  let dummy = Node {val: 0, next: null} in
  let new_tail = ref dummy in
  let cur_tail = !Tail in
  cur_tail.val := v;
  cur_tail.next := new_tail;
  Tail := new_tail

(* methods used by "dequeuer" thread: *)
let deque () =
  let cur_head = !Head in
  if (cur_head == !Tail) then None
  else
    let v = cur_head.val in
    OldHeadVal := v;
    Head := cur_head.next;
    let to_free =
      if (cur_head == !BeingRead) then
        let old_read = !FreeLater in
        FreeLater := cur_head;
        old_read
      else cur_head
    in free to_free; Some v
let read_head_d () =
  let cur_head = !Head in
  if (cur_head == !Tail) then None
  else Some cur_head.val

```

(a) Original implementation by Jayanti and Petrovic [22] expressed in $\lambda_{\text{ref}}^{\text{conc}}$

```

let enqueueer v =
  if (v == ()) then
    Some (read_head_e ())
  else
    match (to_int v) with
    | Some n => Some (enqueue n)
    | None => None
end

let dequeuer v =
  if (v == true) then
    Some (read_head_d ())
  else if (v == false) then
    Some (deque ())
  else
    None

```

(b) Wrappers to be verified

■ **Figure 7** Single-enqueueer, single-dequeueer queue

483 we have added to $\lambda_{\text{ref}}^{\text{conc}}$ for this example), the semantics of which are as follows:

484 $\text{to_int}(n \in \mathbb{Z}) \xrightarrow{\text{pure}} \text{Some } n \quad \text{to_int}(v \notin \mathbb{Z}) \xrightarrow{\text{pure}} \text{None}$

485 Restricted wait-freedom is stated as a variant of Definition 3, with changes highlighted:⁵

486 ► **Definition 7** (Wait-freedom). *Given a set of “initialized” configurations $C \subseteq \text{Config}$, we*
 487 *define restricted wait-freedom of list of operations $\text{ops} : \text{List}(\text{Val})$ as*

$$\begin{aligned}
 & \text{waitFreeRestr}_C(\text{ops}) \triangleq \\
 & \forall \text{etr}. \text{length}(\text{etr}[0].1) = \text{length}(\text{ops}) \wedge \\
 & \left(\bigwedge_{\tau \mapsto e \in \text{etr}[0].1} \exists e'. \text{etr}[0].1[\tau] = e'[\text{ops}[\tau]/x] \wedge \text{noLocs}(e') \wedge \text{noForks}(e') \right) \wedge \\
 & \text{etr}[0] \in C \Rightarrow \forall \tau. \text{alwaysReturnsInTrace}(\text{ops}[\tau], \text{etr}, \tau)
 \end{aligned}$$

492 This definition establishes a property of a *list* of methods—one operation may appear
 493 multiple times in this list. Initially (at the start of etr), we have $\text{length}(\text{ops})$ many threads,
 494 and the i^{th} thread is a client of the i^{th} method in ops . Importantly, none of threads

⁵ Note that below we assume that $\text{Thread} \triangleq \mathbb{N}$ and that a thread pool can be treated as a list of expressions.

XX:16 Verifying wait-freedom for concurrent higher-order programs

threads should fork any other threads—otherwise the restriction on the number of concurrent operations might be violated. The *alwaysReturnsInTrace* conclusion states that every call to one of the operations in any of the threads must always return.

For example, restricted wait-freedom of the algorithm in Figure 7b is stated using Definition 7 by taking $\text{ops} \triangleq [\text{enqueueer}, \text{dequeueer}]$. A more complicated n -enqueueer, single-dequeueer algorithm (also presented in [22]) requires to use a list consisting of n enqueueer methods and a single dequeueer method, *i.e.*, $\text{ops} \triangleq \text{dequeueer} :: \text{repeat}(n, \text{enqueueer})$.

Verifying restricted wait-freedom requires **changes** to the specification we had given in Definition 4. We will first give the formal definition and explain the details afterwards.

► **Definition 8** (Lawyer specification of restricted wait-freedom). *For a set of initialized configurations $C \subseteq \text{Config}$, we define the Lawyer specification of $\text{ops} : \text{List}(\text{Val})$ restricted wait-freedom as follows, for some $\mathbf{F} : \mathbb{N}$:*

$$\begin{aligned} \text{WaitFreeRestrSpec}_C(\text{ops}) &\triangleq \forall c \in C. \text{heap_repr}(c) \Rightarrow \text{mtoks}(\text{ops}) * \\ &\quad \bigstar_{\text{op} \in \text{ops}} \left(\forall \tau, \pi, a. \{ \mathbf{F} \cdot \text{bar}_\circ \pi * \text{ph}_\circ \tau \pi * \text{mtok}(\text{op}) \} \text{ op } a \{ _ . \text{ph}_\circ \tau \pi * \text{mtok}(\text{op}) \}^\tau \right) * \\ &\quad \bigstar_{\text{op} \in \text{ops}} \left(\forall \tau, a. \{ \text{mtok}(\text{op}) \}^\sharp \text{ op } a \{ v. \text{isGroundVal}(v) * \text{mtok}(\text{op}) \}^\tau \right) \\ &\quad \text{where } \text{mtoks}(l : \text{List}(\text{Val})) \triangleq \text{toks}_{\text{ops}}(l) \quad \text{and} \quad \text{mtok}(m : \text{Val}) \triangleq \text{mtoks}([m]) \end{aligned}$$

In the definition above, $\text{isGroundVal}(v : \text{Val})$ states that the value v does not contain any locations or lambda functions. This is related to the restriction where we require the queue to only store integers which we will discuss in Section 7. The proposition $\text{mtoks}(l)$ (an abbreviation for $\text{toks}_{\text{ops}}(l)$ ⁶) asserts ownership of the list l of *method tokens*. Similarly to the tokens used in Section 3, these are created upon the module initialization and have to be provided upon every call to the corresponding operation; the difference is that a method token is given back after the return to allow subsequent calls. Method tokens satisfy the following laws (where $\text{count}(m, l)$ counts occurrences of m in l):

$$\text{mtoks}(l_1) * \text{mtoks}(l_2) \dashv\vdash \text{mtoks}(l_1 ++ l_2) \quad \text{mtoks}(l) \vdash \forall m \in l. \text{count}(m, l) \leq \text{count}(m, \text{ops})$$

Finally, the adequacy theorem for restricted wait-freedom is similar to Theorem 5:

► **Theorem 9** (Adequacy for restricted wait-freedom). *If $\text{WaitFreeRestrSpec}_C(\text{ops})$ is provable in Lawyer, then $\text{waitFreeRestr}_C(\text{ops})$ holds in the meta-logic.*

We use Theorem 9 to establish $\text{waitFreeRestr}_{C_Q}([\text{enqueueer}, \text{dequeueer}])$ (where C_Q is some set of allowed initial states for the queue in Figure 7a) by verifying the wrappers in Figure 7b. We note that despite the fact that termination of queue methods follows almost immediately—they all terminate in constant time—proving their safety is highly non-trivial: reading of the head node might execute concurrently with dequeuing that involves freeing the head. We refer the reader to our Rocq development for the details of the proof.

6 Proving the wait-freedom adequacy theorem

As Section 4 demonstrates, proving the specification in Definition 4 for a given operation is relatively straightforward. Indeed, a wait-free operation cannot block, or be delayed by

⁶ In fact, the tokens resource $\text{toks}_N(k)$ from Figure 4 can be defined as $\text{toks}_{\text{ops}}(l)$ where l and ops are lists of unit values of length k and N correspondingly.

other operations running concurrently, thus it must terminate in finite number of steps. To account for this, the rules of Lawyer logic simply consume one fuel resource upon every step; essentially, the user is mainly responsible for providing enough fuel initially. This applies both to the original specification in Definition 4, and its variants described in Section 5. Thus, the non-trivial parts of the proof might only concern safety of the operation.

However, showing that the specification in Definition 4 implies wait-freedom, *i.e.* proving Theorem 5, is quite challenging. Ideally, we would just reuse the existing work on termination verification in Lawyer for proving wait-freedom. That is, we would ideally reuse Lawyer's proof that verifying a program in Lawyer establishes a refinement between the program and an always terminating transition system, which then guarantees that the program terminates. We might hope that this would help us derive termination of all calls to our wait-free operation. However, we face the following challenges:

- C1 The property established by Lawyer's adequacy theorem is termination of closed programs, *i.e.*, in our case the client program together with the wait-free data structure. In our case, we aim to prove termination of each individual call to the wait-free operation. The latter property would follow from the former, if the client program is terminating, but the client program does not have to be terminating in general.
- C2 Lawyer (and Trillium in general) requires the entire closed program to be verified for the adequacy theorem to apply, and to allow us to derive a property of its execution traces. For wait-freedom, we are given an arbitrary client program (that uses the wait-free operation) and cannot verify it directly.
- C3 The Lawyer logic does not contain rules that support verifying non-terminating programs. For example, there are no rules that allow generating fuel indefinitely long to be spent in an infinite execution.

We address these challenges as follows. First, in Section 6.1 we reduce the problem of proving wait-freedom to showing termination of the closed program. With that, we are able to reuse the Lawyer's adequacy theorem, thus addressing the Challenge C1. Then, in Section 6.2 we employ the well-known method of logical relations to establish "robust safety" of the wait-free operation, *i.e.*, we prove that an arbitrary client does not violate operation's internal invariants. This solves the Challenge C2. Finally, in Section 6.3 we show that the aforementioned reduction in fact allows to apply the Lawyer logic to generate fuel consumed by an infinitely running program. Together with the robust safety result, it allows to verify a non-terminating program, which solves Challenge C3.

Finally, we note that Theorem 5 and its variants for possibly-stuck wait-freedom from Section 5.1 are proven in the same way with minimal variations. However, proving Theorem 9 requires non-trivial changes to the logical relation and the way it is used. For space reasons, we refer the reader to technical Rocq development for that. Therefore, the rest of the section only explains how the original Theorem 5 is proven.

6.1 Reducing wait-freedom to termination

The first step in proving the wait-freedom adequacy theorem is to reduce proving wait-freedom to showing termination of the closed program. For that, the proof of Theorem 5 proceeds via a proof by contradiction. Namely, for the operation op and some set of initialized configurations $C \subseteq \text{Config}$ we assume that an execution trace etr , evaluation context E , thread τ and trace index i such that:

- $\text{WaitFreeSpec}_C(\text{op})$ is provable in Lawyer.

- 568 ■ The premises of both $\text{waitFree}_C(\text{op})$ and $\text{alwaysReturnsInTrace}(\text{op}, \text{etr}, \tau)$ hold. In particular, a call to op occurs in etr at index i , thread τ under evaluation context E .
- 570 ■ Yet, the conclusion of $\text{alwaysReturnsInTrace}(\text{op}, \text{etr}, \tau)$ (i.e. $\exists j \geq i. \text{Ret}(\text{etr}[j].1, \tau, E)$) does not hold.

572 Note that the only way this is possible is in the case where etr is infinite. Indeed, if the
 573 call to op never returns, then by the schedUntilRet assumption of $\text{alwaysReturnsInTrace}$ the
 574 calling thread τ is always eventually scheduled. We will prove the contradiction by showing
 575 that the same set of assumptions simultaneously implies that etr must terminate.

576 To prove that etr terminates, we refine it to a trace of the Obligations Model of Lawyer;
 577 the adequacy theorem of Lawyer implies the finiteness of both traces. For establishing the
 578 refinement, we need to verify the program (i.e., each element of the expressions in the starting
 579 thread pool, $\text{etr}[0].1$). As mentioned in Challenge C2 and Challenge C3, the Lawyer logic
 580 can only support terminating programs, whereas we already concluded above that etr is
 581 infinite, and moreover we cannot apply the rules of Lawyer directly, as the program to verify
 582 (a client of the wait-free operation) is universally quantified over.

583 The key to establishing the refinement is exploiting the assumptions above. Indeed, in
 584 Section 6.3 we show how to reuse Lawyer logic and construct a refinement *assuming* the
 585 premises above and given the specification for the wait-free operation. In other words, we
 586 establish the refinement *only* for execution traces that satisfy the premises above. This is in
 587 contrast with the adequacy theorem of Lawyer (and Trillium) which establish the refinement
 588 for *all* execution traces. Therefore, we need to generalize the adequacy theorem of Trillium to
 589 establish a refinement only when some condition on execution holds. In return, the fact that
 590 this condition holds is propagated to the refinement construction via user-provided “progress
 591 resource” described in more detail in Section 6.3. Below, we provide the generalized adequacy
 592 theorem of Trillium and [highlight](#) the main changes compared to the original theorem.

593 ► **Theorem 10** (Conditional adequacy theorem of Trillium). *Consider a transition system $\mathcal{M}$
 594 and a relative image-finite relation ξ on finite traces of $\lambda_{\text{ref}}^{\text{conc}}$ and the model. Then, consider
 595 $\text{TI} : \text{FinTrace}(\lambda_{\text{ref}}^{\text{conc}}) \rightarrow \text{FinTrace}(\mathcal{M}) \rightarrow i\text{Prop}$ — a representation of the aforementioned
 596 finite traces in the Iris logic. Then, consider a predicate F on finite execution traces. Finally,
 597 consider a “progress resource” $\text{PR}_F : \text{FinTrace}(\lambda_{\text{ref}}^{\text{conc}}) \rightarrow i\text{Prop}$ that satisfies a number of
 598 laws described in [appendix\[2\]](#).*

599 *Let etr be an execution trace and $\mathbf{m}$ a state of $\mathcal{M}$. If $\xi(\text{etr}[0], \mathbf{m})$ holds (treating its
 600 arguments as singleton traces), $F(\text{etr}')$ holds for any finite prefix etr' of etr , and furthermore
 601 we have (again, treating $\text{etr}[0]$ and $\mathbf{m}$ as singleton traces)*

$$602 \quad \vdash \models \text{TI}(\text{etr}[0], \mathbf{m}) * \text{AlwaysHolds}(\xi, \text{etr}[0], \mathbf{m}) * \text{PR}_F(\text{etr}[0])$$

603 *then there exists such model trace mtr that $\text{mtr}[0] = \mathbf{m}$ and $\hat{\xi}(\text{etr}, \text{mtr})$ holds.*

604 We refer the reader to Trillium [37] for definitions of AlwaysHolds and $\hat{\xi}$; the latter is
 605 essentially the refinement we’re looking for. Intuitively, the three resources in the Iris premise
 606 of Theorem 10 correspondingly describe *what* is the current state of the execution, *why* this
 607 state representation implies refinement at the meta-level, and *how* this representation is
 608 preserved on every step of the execution.

609 We prove the finiteness of our etr by applying Theorem 10 and establishing the refine-
 610 ment to the Obligations Model. For that, we choose the trace interpretation, the starting
 611 model state $\mathbf{m}$, and the refinement relation that the Lawyer’s adequacy theorem does when
 612 it uses Trillium’s adequacy theorem (the latter being slightly adjusted — see our Rocq

implementation for details). We defer the choice of the predicate F and the progress resource to Section 6.3; we only note that the chosen F indeed holds for all prefixes of etr . Thus, it remains to prove the Iris premise of Theorem 10. There, $\text{TI}(etr[0], \mathbf{m})$ and $\text{AlwaysHolds}(\xi, etr[0], \mathbf{m})$ are proved similarly to how it is done in Lawyer. We will explain how to establish the progress resource in Section 6.3. Thus, using the Trillium's adequacy theorem we prove that etr is finite, and thus derive a contradiction.

6.2 Logical relation for ensuring “robust safety” of a wait-free operation

As the Challenge C2 states, using a Lawyer-based approach requires us to verify the entire program under consideration, including the arbitrary and thus unknown client of the operation. Moreover, as the Challenge C3 states, the Lawyer logic in general does not support verification of non-terminating programs which we would like to consider as clients of wait-free operations.

However, it turns out that verifying the operation's client can be split into two steps. The first step is proving the *robust safety* of the operation, *i.e.* the fact that any operation's client preserves operation's internal invariants. The next step, described further in Section 6.3, is showing that any execution of a client refines the Obligations Model of Lawyer.

To prove robust safety, we employ the well-known idea of logical relations [36, 14]. This approach is based on proving, for a given expression e , an Iris proposition $\text{RS}_{\mathbf{E}}(e)$; the predicate $\text{RS}_{\mathbf{E}}(\cdot)$ is called the *expression relation*. The expression relation intuitively captures that any execution of e preserves all invariants in the system (including those of the wait-free operation), but it may get stuck. Crucially, the expression relation is proven to be closed under program compositions, *e.g.*, if both e_1 and e_2 are in the expression relation, so is the expression $e_1 \ e_2$, *i.e.*, calling the function e_1 with argument e_2 —intuitively, the worst that can happen here is e_1 not a function in which case $e_1 \ e_2$ gets stuck, which is allowed. This compositionality of the expression relation is the reason why to establish robust safety of the operation it is sufficient to prove $\text{RS}_{\mathbf{E}}(e)$ for all clients e of the operation. In Section 4 we described a PresInv Hoare triple that states a property similar to $\text{RS}_{\mathbf{E}}(\cdot)$. Indeed, expression relation is defined similarly to PresInv (we explain $\text{RS}_{\mathbf{V}}(v)$ below):

$$\text{RS}_{\mathbf{E}}(e) \triangleq \forall \tau. \{\text{True}\}^{\sharp} e \{v. \text{RS}_{\mathbf{V}}(v)\}^{\tau}$$

To understand how $\text{RS}_{\mathbf{E}}(e)$ is proven, first consider some e that does not actually call the wait-free operation. The potentially unsafe steps of e execution are writing to memory location (or freeing it) or evaluating an arbitrary, potentially unsafe lambda expression. For these steps to be safe, we need to ensure that the corresponding location or lambda expression are also “safe to use”. More generally, we introduce the following *value relation* $\text{RS}_{\mathbf{V}}(v)$:⁷

► **Definition 11** (Value relation).

$$\begin{aligned} \text{RS}_{\mathbf{V}}(()) &\triangleq \text{RS}_{\mathbf{V}}(b : \mathbb{B}) \triangleq \text{RS}_{\mathbf{V}}(n : \mathbb{Z}) \triangleq \text{True} \\ \text{RS}_{\mathbf{V}}(\text{inl } v) &\triangleq \text{RS}_{\mathbf{V}}(\text{inr } v) \triangleq \text{RS}_{\mathbf{V}}(v) \\ \text{RS}_{\mathbf{V}}((v_1, v_2)) &\triangleq \text{RS}_{\mathbf{V}}(v_1) * \text{RS}_{\mathbf{V}}(v_2) \\ \text{RS}_{\mathbf{V}}(\ell : \text{Loc}) &\triangleq [(\exists v : \text{Val}. \ell \mapsto v * \text{RS}_{\mathbf{V}}(v)) \vee \text{freed}(\ell)] \\ \text{RS}_{\mathbf{V}}(\text{rec } f \ x := e) &\triangleq \forall \tau, v. \{\text{RS}_{\mathbf{V}}(v)\}^{\sharp} (\text{rec } f \ x := e) v \{r. \text{RS}_{\mathbf{V}}(r)\}^{\tau} \end{aligned}$$

⁷ The resource $\text{freed}(\ell)$ is obtained after executing $\text{free } \ell$ and is irrelevant for the rest of the paper.

651 Note that the last two cases of this definition reference $RS_V(v')$ for some value v' which
 652 might be structurally larger than one being considered. The soundness of such definitions is
 653 guaranteed by the guarded recursion of Iris; we refer the reader to [24] for more details.

654 Thus, proving $RS_E(e)$ for an expression e that does not call the wait-free operation
 655 involves keeping track of the values produced by executing e so far and making sure they
 656 all satisfy the value relation. With that, if e does not contain hard-coded locations initially
 657 and does not use pointer arithmetic, we can indeed prove $RS_E(e)$ by structural induction⁸,
 658 showing that evaluating its sub-expressions produce values that can be safely used afterwards.

659 However, an e that actually calls a wait-free operation op requires some extra work. Indeed,
 660 values produced during op execution might not satisfy the value relation (*e.g.* locations
 661 used by op internally are covered by op 's own invariants), and also op might use pointer
 662 arithmetic (*e.g.* queue in Figure 7a uses it for accessing `Node` structure fields). Thus, we
 663 cannot automatically derive expression relation for calls to op —instead, the user must show it.
 664 And indeed, the $PresInv(op)$ part of op wait-free specification is exactly the lambda-expression
 665 case of Definition 11, stating that it is always safe to call op with any argument. Another
 666 way to think about this is that showing $PresInv(op)$ is also required because it ensures that
 667 the operation does not expose its own internal state, which would of course not be safe.

668 A more general fact is stated by the “fundamental theorem” of logical relations. Namely,
 669 that the expression relation holds for any expressions whose free variables are substituted
 670 with values that are themselves in the value relation:

671 ► **Theorem 12** (Fundamental theorem). *Let e be an expression that contains neither any `Loc`*
 672 *literals nor any pointer arithmetic, and whose free variables are $\vec{x}s$. Given any list of values*
 673 *$\vec{v}s$ such that $|\vec{v}s| = |\vec{x}s|$. Then the following holds:*

$$674 \quad \left(\bigstar_{v \in \vec{v}s} RS_V(v) \right) \vdash RS_E(e[\vec{v}s/\vec{x}s])$$

675 An op 's client can be represented as a thread pool made of expressions into which op is
 676 substituted (see the first premise of Definition 3). To prove robust safety of op , we apply
 677 Theorem 12 and provide $PresInv(op)$ to satisfy its premise. Thus, we establish the following:

678 ► **Theorem 13** (Robust safety of the wait-free operation). *Let $\mathcal{E}$ be a thread pool such that its*
 679 *every element e has a single free variable x , and contains neither `Loc` literals, nor any pointer*
 680 *arithmetic. Assume that $WaitFreeSpec_C(op)$ holds for some C . Then the following holds:*

$$681 \quad \vdash \bigstar_{e \in \mathcal{E}} RS_E(e[op/x])$$

682 6.3 Refining an infinite execution to a trace of Obligations Model

683 In Section 6.1, we have reduced proving wait-freedom to that of termination, which we
 684 establish by applying Theorem 10 and refining a given execution to a trace of Obligations
 685 Model of Lawyer. However, that reduction, by contradiction, considers an *infinite* execution,
 686 whereas the main property of Obligations Model is the finiteness of its traces. As the
 687 Challenge C3 states, the Lawyer logic that establishes refinement to the Obligations Model
 688 was designed to only verify terminating programs. Here, we briefly describe the solution that

⁸ For lambda expressions, it also involves so-called Löb induction available in Iris-like step-indexed logics.

allows to repurpose the Lawyer logic for verifying non-terminating programs. Due to lack of space we refer the reader to our Rocq formalization [1] for the details.

The central part in establishing refinement with Theorem 10 is the progress resource. Intuitively, the resource $\text{PR}_F(etr)$ means that if the finite execution trace etr is currently refined by some trace of the Obligations Model, and if it takes a step into a new trace etr' such that $F(etr')$ holds, then etr' will also refine some Obligations Model trace, all the invariants are preserved, and $\text{PR}_F(etr')$ is reestablished. Thus, the progress resource for a singleton trace $\text{PR}_F(c)$ essentially means that any execution starting from the configuration c is refined by a trace of the Obligations Model, as long as it does not violate the predicate F .

Crucially, the user is free to choose an arbitrary progress resource, as long as it satisfies a number of laws listed in the appendix [2]. The original adequacy theorem of Lawyer can be stated as a corollary of Theorem 10 where the progress resource ignores the predicate F , and for the starting configuration it consists of Hoare triples for every thread present in that configuration. In our case, we cannot prove such triples, as we consider non-terminating client programs, and the Lawyer logic is designed to establish termination.

Thus, the progress resource for wait-freedom consists of two parts: one that guarantees preservation of invariants and another that ensures the refinement of the Obligations Model. The former is given by the robust safety result of Theorem 13; the latter is more elaborate.

The main part for establishing a refinement of the Obligations Model is providing a barrel of fuel on every execution step. For that, we exploit the assumptions on the execution that were obtained during the reduction in Section 6.1. Namely, that our progress resource is defined as $F(etr) \triangleq \text{infCallPrefix}(E, \tau, i, etr)$, where E, τ and i are fixed during the reduction. We refer the reader to Rocq development [1] for its formal definition. Intuitively, this predicate means that etr is a possible finite prefix of some infinite trace in which thread τ has an infinite call to `op` under evaluation context E at index i . Now, knowing i , we can pre-allocate i barrels of fuel in the initial progress resource, so that the refinement can be preserved for the first i steps before the infinite call.

To provide fuel for the rest of execution, we first note that the thread τ that runs that call does not need infinite fuel: the specification $\text{NoInfExec}(\text{op})$ states that any call to `op` refines a trace of the Obligations Model, as long as a fixed amount of fuel is provided to satisfy the precondition of that specification. Therefore, we add this fuel to the initial progress resource.

To provide fuel to all other threads, we employ a trick that allows to *generate fuel indefinitely*. That is, we use the Lawyer logic's obligations mechanism originally designed for verifying blocking concurrency. We refer the reader to Lawyer [33] for more details; in short, threads can wait for other threads' actions by generating fuel, as long as this waiting is not circular. In our case, we can assign the calling thread τ an obligation that is never fulfilled which does not correspond to any action, but merely enables the fuel generation mechanism. This lets all other threads to wait for this obligation to be fulfilled. By assumption, τ never terminates, and thus the other threads can always receive fuel. Of course, this trick is only possible because we started the proof in Section 6.1 by contradiction—under normal circumstances, a thread with an obligation eventually fulfills it, which disables fuel generation.

7 Discussion and limitations

7.1 Restricting the content of wait-free data structures

When verifying the queue implementation in Section 5.2, we introduced a seemingly unnecessary constraint. Namely, the wrapper `enqueueer` in Figure 7b only allows the integer values to be enqueued. Note that the original implementation in Figure 7a never inspects

the enqueued values and thus does not rely on this constraint. Yet, in short, lifting this constraint would imply adding an extra premise to the precondition of `NoInfExec` which must hold for the argument — but since we show termination of calls to an operation with *any* argument, we would not be able to prove it when using `NoInfExec` in the adequacy proof.

Before proceeding to details of this constraint, we explain its scope. Despite arising when verifying restricted wait-freedom in Section 5.2, the issue is not related to tokens, or the restricted wait-freedom in general. However, it arises with all data structures that contain methods that return values previously stored by other methods (queues, stacks, linked lists *etc.*). In our context, such structures happen to be restricted wait-free.

Recall that establishing robust safety of `dequeuer` in Figure 7a using Theorem 13 requires proving the `PresInv(dequeuer)` specification, which is the lambda-expression case of Definition 11. (For technical reasons, Definition 8 requires a stronger specification from which `PresInv(dequeuer)` is derivable, but the issue would still hold without this stronger specification.) Note that when `dequeuer` returns a dequeued value, it cannot, by itself, prove that it is safe-to-use. For this to hold, the queue invariant must enforce that all values stored are safe-to-use by storing the $RS_V(v)$ resources for every stored value v . To maintain this invariant, `enqueueer` must only call `enqueue` with safe-to-use values — and it indeed does, as only integers are being enqueued.

If we remove the only-integers constraint, the specifications for `enqueueer` would have to include the $RS_V(v)$ premise in the precondition. In particular, it would appear in the precondition of `NoInfExec(enqueueer)`. When proving wait-freedom, we consider a potentially non-terminating call to `enqueueer` with some argument a . Internally, the adequacy theorem uses (the token-based version of) `NoInfExec(enqueueer)` to verify the call `enqueueer a`. Since $RS_V(a)$ is not provable in general for arbitrary a , we would not be able to use this specification.

To lift this limitation, one has to exploit the fact that a client program satisfying the conditions of Theorem 13 can only produce safe-to-use values as it executes (except when the wait-free operation is in progress). We leave establishing this formally to future work.

7.2 Proving linearizability of wait-free data structures

An important criteria for the correctness of a concurrent data structure is *linearizability* [19]. A data structure is linearizable if whenever operations on that structure are run concurrently, each of them appear to take effect atomically, even if they actually take multiple steps to finish. Linearizability makes it easier to reason about the data structure as one does not have to consider all possible interleavings of the operations—one can simply reason as though these operations happen one after another. It is thus not surprising that many wait-free data structures are designed to be linearizable [22, 28, 38, 15]. Therefore, it is desirable to verify both wait-freedom and linearizability for wait-free data structure.

In fact, proving linearizability with program logics is well-studied [21, 9, 26, 5]. Birkedal *et al.* [5] have formally shown that linearizability is internalized by specifications given in terms of the so-called *logically atomic Hoare triples* [26]. Moreover, the Lawyer approach [33] that we build upon provides a liveness-aware version of the logically atomic specification. However, even simple wait-free implementations contain what is known as *future-dependent linearization points* [39] where the point at which linearization takes place within an operation can depend on how the other operations started afterwards execute. Moreover, the linearization point of an operation might happen inside another operation that runs concurrently with it in a different thread. It is the case for the queue in Figure 7a. Consider the scenario where the dequeuer thread runs a number of `dequeue` operations one by one, while the enqueueer thread is running the single `read_head_e` operation. In that case, the linearization point of

782 `read_head_e`, if the comparison after the second read of `Head` fails, will occur *when the last*
 783 *dequeue operation writes to OldHeadVal.*

784 Iris supports verifying future-dependent linearizability with using so-called prophecy
 785 variables [25]. However, as is, Lawyer does not support prophecy variables because Trillium
 786 [37] does not. Thus, verifying both wait-freedom and linearizability in our framework would
 787 require extending the Lawyer logic and the Trillium logic first.

788 7.3 Wait-free operations of a lock-free data structure

789 Some data structures provide both lock-free and wait-free methods. For example, an
 790 implementation of counter can include a lock-free increment from Figure 1b and a wait-free
 791 `read` method that simply reads the current counter value in a single step. Thus, in general,
 792 wait-freedom is a property of a single method, not an entire data structure. Our approach
 793 could also be used for verifying wait-freedom in such a setting as well. First, the meta-level
 794 definition of wait-freedom should mention both a wait-free operation, and a number of
 795 lock-free operations that may run concurrently with it (similarly to how Definition 7 applies
 796 to the list of wait-free operations). As for the specifications, the approach suggested by Total
 797 TaDA [10] applies: the operations running concurrently with the wait-free one only need to
 798 preserve invariants. With that, we can prove a generalization of Theorem 5 that establishes
 799 wait-freedom of an operation `op` given `WaitFreeSpec(op)` and proofs of `PresInv(op')` for every
 800 lock-free operation `op'` running concurrently with `op`.

801 8 Related work

802 All prior solutions verify wait-freedom for first-order languages. These solutions assume,
 803 in one way or another, that the client of a wait-free operation is safe, *i.e.* that it does not
 804 violate the data structure's internal invariants. Instead, we prove once and for all that all
 805 possible clients in our higher-order programming language satisfy this property via a logical
 806 relations argument as explained in Section 6.2. Moreover, most of the prior solutions only
 807 support unrestricted wait-freedom, lack case studies, and are not mechanized.

808 8.1 Program logics for wait-freedom

809 **Total TaDA** The program logic of [10] supports verification of non-blocking program
 810 properties, including unrestricted wait-freedom. Proving a program specification in that
 811 logic establishes a meta-level judgement about program execution. This judgement is defined
 812 as a least fixed point and thus guarantees the program's termination. The non-trivial
 813 (from liveness perspective) rules of the Total TaDA logic allow verification of while loops
 814 and recursive functions by choosing a decreasing measure of termination. Moreover, their
 815 specifications are capable of expressing linearizability, which we do not consider. This
 816 approach is demonstrated by verifying wait-freedom of `read` operation of a counter.

817 Total TaDA assumes that the operation's client is safe by employing a rely relation [23]
 818 into the aforementioned meta-level judgement established for the operation. The only way
 819 to ensure that the client respects the rely relation is to verify it; thus their definition of
 820 wait-freedom does not quantify over arbitrary client as ours does. However, they support
 821 clients verified with the non-total TaDA [9] logic. Contrary to that, our approach supports
 822 arbitrary client programs (up to assumptions of Theorem 12). Total Tada is not mechanized.

Lili The approach of [30] verifies liveness of both blocking and non-blocking operations of concurrent objects, as well as their linearizability. In that work, (unrestricted) wait-freedom is treated as starvation-freedom (a property usually applied to blocking algorithms) without the fair scheduling assumption. The Lili logic is parametric, and by instantiating it in a specific way one can exclude rules that justify threads being delayed and/or blocked by each other. This is similar to how Lawyer proves *e.g.* termination under unfair scheduler with a more restrictive instantiation of Obligations Model.

The adequacy statement of Lili is similar to ours. It states that for an arbitrary execution trace starting from an arbitrary client program configuration, and an “appropriate” state, all calls to the object’s methods terminate. However, the major difference compared to our approach is that the physical state of the concurrent object and its client are explicitly separated. That way, in Lili the invariants of the concurrent object are trivially preserved by its clients who do not have access to them at all. In our work, our higher-order language does not allow us easily state and use such a separation, and thus we use our logical relations model to prove safety of arbitrary client.

The Lili logic is neither higher-order nor mechanized. Moreover, neither Lili, nor the lock-free fragment of its predecessor [31] have any case studies on wait-freedom verification.

Automatic tool for verifying non-blocking algorithms The work of [16] verifies progress properties of non-blocking algorithms. They notice that progress of these algorithms can be established by identifying a set of ordered abstract actions. An action at a higher level cannot be executed infinitely often unless there is an action of lower level which is also executed infinitely often. In this setting, (unrestricted) wait-freedom corresponds to discrete orders, *i.e.* no action can ever be executed infinitely often. In the absence of non-trivial orders between actions and unbounded **while** loops, Gotsman *et al.* [16] reduce verifying wait-freedom to verifying safety.

This solution relies on an automatic tool that establishes safety of a given operation. Moreover, it establishes the rely relation that the operation’s client is expected to preserve. Similarly to Total TaDA, this means that wait-freedom is only established for verified clients.

Gotsman *et al.* [16] present no wait-freedom case studies. They explain that their approach might not apply to non-trivial wait-free operations due to the limitations of their tool. As an example, they mention the wait-free **contains** operation of the concurrent set algorithm by [40]. The **while** loop in that operation traverses a sorted list of integers up to a given number. As the list can be concurrently modified during the traversal, termination of such loop relies on the list being sorted. The tool by [16] cannot exploit this fact, whereas in our approach the termination of such loops can be shown by providing the amount of fuel proportional to the given number.

Moreover, their tool has limited safety reasoning capabilities and thus cannot verify examples like Figure 7a where the safety argument is highly non-trivial. The tool is automated, but the presented theory is not mechanized in a proof assistant.

Wait-freedom of “fast-path-slow-path” in VST The work of [34] considers non-blocking algorithms that are instances of a specific variation of the “fast-path-slow-path” approach [28]. This approach aims to design efficient wait-free data structures by combining a lock-free implementation (which is often faster, but does not have wait-free progress guarantee) and a wait-free implementation (slower, but has guaranteed progress).

The authors of [34] establish a pen-and-paper proof that such algorithms, if safe, are indeed wait-free, and provide an extension of the VST logic [3] to verify their safety, and thus

also wait-freedom. This approach limits the number of concurrent threads, and therefore it can only be used to establish restricted wait-freedom. Similar to Total TaDA and Gotsman *et al.* [16], such approach requires the user to verify the entire program.

As a case study, Peterson *et al.* [34] verify wait-freedom of the queue by Kogan and Petrank [28], which is in turn an optimization of the queue by Kogan and Petrank [27]. In the implementation by [28], a thread helps up to N other concurrent operations before performing its own operation (where N is the maximal number of threads). We believe that our approach can also support verifying this case study by requiring an amount of fuel proportional to N .

8.2 Universal constructions

Designing a wait-free implementation for every possible data structure is tedious, and therefore a question arises as to whether it is possible to design a universal wait-free construction. And indeed, that is possible. These are algorithms that turn any *sequential* data structure implementation into a (restricted) wait-free one. The early universal constructions such as [18] are considered impractical due to their high time and space complexity of the resulting wait-free implementations. Luckily, this is less of an issue for more recent constructions mentioned below. However, none of these constructions has a mechanized proof of wait-freedom.

Therefore, it is interesting whether our solution can be used to verify such constructions. To do so, we would need to prove a specification for the sequential implementation, and use it together with a general proof of the universal construction in question — a process similar to how we verified possibly-stuck wait-freedom of `list_map(incr l)` in Section 5.1. Therefore, we believe that our approach can be applied to universal constructions as well.

Examples of modern wait-free universal constructions include P-Sim [12] (along with its extensions such as [13]) and CX [8]. The time complexity of these approaches is proportional to the number of concurrent operations, and the execution time of the provided sequential implementation. Given that these parameters are known, we believe that our approach can verify restricted wait-freedom of the resulting implementations, as the required amount of fuel is known in advance.

9 Conclusion and future work

In this work, we have presented the first solution to verifying wait-freedom in a higher-order language. The key idea is internalizing the notion of wait-freedom in the Lawyer separation logic with a particular specification pattern. To establish this connection formally we presented a new adequacy theorem for Lawyer specifically for programs proven against a spec in the aforementioned pattern. We also used a logical relations model to prove that a wait-free operation can be safely used by an arbitrary, higher-order client. We demonstrated our approach by verifying a number of examples, with the more complicated of them requiring a slightly different notion of wait-freedom which we call restricted wait-freedom, and to accordingly adjust the wait-freedom specification pattern. Our solution and case studies are all mechanized in the Rocq Prover.

There are multiple notable directions of future work. First, we aim to extend our approach to verify related properties, such as linearizability and bounded wait-freedom [7]. It would also be interesting to verify one of the state-of-the-art universal constructions for wait-freedom. Finally, we hope to be able to improve the usability of our approach by deriving the two specifications needed by Definition 4 from a single, stronger specification that implies both.

913 — References —

- 914 1 Anonymous. Supplementary rocq development for the wait-freedom paper, 2026.
- 915 2 Anonymous. Technical appendix for the wait-freedom paper, 2026.
- 916 3 Andrew W. Appel. Verified software toolchain. In Gilles Barthe, editor, *Programming*
917 *Languages and Systems*, pages 1–17, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- 918 4 Lars Birkedal and Ales Bizjak. Lecture notes on Iris: Higher-order concurrent separation logic,
919 2017. URL: <http://iris-project.org/tutorial-pdfs/iris-lecture-notes.pdf>.
- 920
- 921 5 Lars Birkedal, Thomas Dinsdale-Young, Armaël Guéneau, Guilhem Jaber, Kasper Svendsen,
922 and Nikos Tzevelekos. Theorems for free from separation logic specifications. *Proc. ACM*
923 *Program. Lang.*, 5(ICFP), aug 2021. doi:10.1145/3473586.
- 924 6 Fabian Bläse. Non-blocking synchronization in operating systems, 2021. URL: https://www4.cs.fau.de/Lehre/WS20/MS-AKSS/arbeiten/paper_blaese_final.pdf.
- 925
- 926 7 Hagit Brit and Shlomo Moran. Wait-freedom vs. bounded wait-freedom in public data
927 structures (extended abstract). In *Proceedings of the Thirteenth Annual ACM Symposium*
928 *on Principles of Distributed Computing*, PODC '94, page 52–60, New York, NY, USA, 1994.
929 Association for Computing Machinery. doi:10.1145/197917.197950.
- 930 8 Andreia Correia, Pedro Ramalhete, and Pascal Felber. A wait-free universal construction
931 for large objects. In *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and*
932 *Practice of Parallel Programming*, PPoPP '20, page 102–116, New York, NY, USA, 2020.
933 Association for Computing Machinery. doi:10.1145/3332466.3374523.
- 934 9 Pedro da Rocha Pinto, Thomas Dinsdale-Young, and Philippa Gardner. Tada: A logic for time
935 and data abstraction. In Richard Jones, editor, *ECOOP 2014 – Object-Oriented Programming*,
936 pages 207–231, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.
- 937 10 Pedro da Rocha Pinto, Thomas Dinsdale-Young, Philippa Gardner, and Julian Suther-
938 land. Modular termination verification for non-blocking concurrency. In Peter Thie-
939 mann, editor, *Programming Languages and Systems - 25th European Symposium on Pro-*
940 *gramming, ESOP 2016, Held as Part of the European Joint Conferences on Theory and*
941 *Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceed-*
942 *ings*, volume 9632 of *Lecture Notes in Computer Science*, pages 176–201. Springer, 2016.
943 doi:10.1007/978-3-662-49498-1_8.
- 944 11 Derek Dreyer, Georg Neis, and Lars Birkedal. The impact of higher-order state and control
945 effects on local relational reasoning. *SIGPLAN Not.*, 45(9):143–156, September 2010. doi:
946 10.1145/1932681.1863566.
- 947 12 Panagiota Fatourou and Nikolaos D. Kallimanis. A highly-efficient wait-free universal con-
948 struction. In *Proceedings of the Twenty-Third Annual ACM Symposium on Parallelism in*
949 *Algorithms and Architectures*, SPAA '11, page 325–334, New York, NY, USA, 2011. Association
950 for Computing Machinery. doi:10.1145/1989493.1989549.
- 951 13 Panagiota Fatourou, Nikolaos D. Kallimanis, and Eleni Kanellou. An Efficient Univer-
952 sal Construction for Large Objects. In Pascal Felber, Roy Friedman, Seth Gilbert, and
953 Avery Miller, editors, *23rd International Conference on Principles of Distributed Systems*
954 *(OPODIS 2019)*, volume 153 of *Leibniz International Proceedings in Informatics (LIPIcs)*,
955 pages 18:1–18:15, Dagstuhl, Germany, 2020. Schloss Dagstuhl – Leibniz-Zentrum für Infor-
956 matik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.OPODIS.2019.18>, doi:10.4230/LIPIcs.OPODIS.2019.18.
- 957
- 958 14 {Aina Linn} Georges. Designing and proving robust safety of efficient capability machine
959 programs, July 2023.
- 960 15 Seep Goel, Pooja Aggarwal, and Smruti R. Sarangi. A wait-free stack. In Nikolaj Bjørner,
961 Sanjiva Prasad, and Laxmi Parida, editors, *Distributed Computing and Internet Technology*,
962 pages 43–55, Cham, 2016. Springer International Publishing.

- 16 Alexey Gotsman, Byron Cook, Matthew Parkinson, and Viktor Vafeiadis. Proving that non-blocking algorithms don't block. *SIGPLAN Not.*, 44(1):16–28, January 2009. doi:10.1145/1594834.1480886.
- 17 M. Herlihy, V. Luchangco, and M. Moir. Obstruction-free synchronization: double-ended queues as an example. In *23rd International Conference on Distributed Computing Systems, 2003. Proceedings.*, pages 522–529, 2003. doi:10.1109/ICDCS.2003.1203503.
- 18 Maurice Herlihy. Wait-free synchronization. *ACM Trans. Program. Lang. Syst.*, 13(1):124–149, January 1991. doi:10.1145/114005.102808.
- 19 Maurice P. Herlihy and Jeannette M. Wing. Linearizability: a correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.*, 12(3):463–492, July 1990. doi:10.1145/78969.78972.
- 20 Bart Jacobs, Dragan Bosnacki, and Ruurd Kuiper. Modular termination verification of single-threaded and multithreaded programs. *ACM Trans. Program. Lang. Syst.*, 40(3):12:1–12:59, 2018. doi:10.1145/3210258.
- 21 Bart Jacobs and Frank Piessens. Expressive modular fine-grained concurrency specification. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '11*, page 271–282, New York, NY, USA, 2011. Association for Computing Machinery. doi:10.1145/1926385.1926417.
- 22 Prasad Jayanti and Srdjan Petrovic. Logarithmic-time single deleter, multiple inserter wait-free queues and stacks. In Sundar Sarukkai and Sandeep Sen, editors, *FSTTCS 2005: Foundations of Software Technology and Theoretical Computer Science*, pages 408–419, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- 23 C. B. Jones. Tentative steps toward a development method for interfering programs. *ACM Trans. Program. Lang. Syst.*, 5(4):596–619, October 1983. doi:10.1145/69575.69577.
- 24 Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.*, 28:e20, 2018. doi:10.1017/S0956796818000151.
- 25 Ralf Jung, Rodolphe Lepigre, Gaurav Parthasarathy, Marianna Rapoport, Amin Timany, Derek Dreyer, and Bart Jacobs. The future is ours: prophecy variables in separation logic. *Proc. ACM Program. Lang.*, 4(POPL), dec 2019. doi:10.1145/3371113.
- 26 Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In Sriram K. Rajamani and David Walker, editors, *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*, pages 637–650. ACM, 2015. doi:10.1145/2676726.2676980.
- 27 Alex Kogan and Erez Petrank. Wait-free queues with multiple enqueueers and dequeuers. *SIGPLAN Not.*, 46(8):223–234, February 2011. doi:10.1145/2038037.1941585.
- 28 Alex Kogan and Erez Petrank. A methodology for creating fast wait-free data structures. *SIGPLAN Not.*, 47(8):141–150, February 2012. doi:10.1145/2370036.2145835.
- 29 Pierre Laborde, Steven Feldman, and Damian Dechev. A wait-free hash map. *Int. J. Parallel Program.*, 45(3):421–448, June 2017. doi:10.1007/s10766-015-0376-3.
- 30 Hongjin Liang and Xinyu Feng. A program logic for concurrent objects under fair scheduling. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '16*, page 385–399, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2837614.2837635.
- 31 Hongjin Liang, Xinyu Feng, and Zhong Shao. Compositional verification of termination-preserving refinement of concurrent programs. In *Proceedings of the Joint Meeting of the Twenty-Third EACSL Annual Conference on Computer Science Logic (CSL) and the Twenty-Ninth Annual ACM/IEEE Symposium on Logic in Computer Science (LICS), CSL-LICS '14*, New York, NY, USA, 2014. Association for Computing Machinery. doi:10.1145/2603088.2603123.

- 1015 32 Maged M. Michael and Michael L. Scott. Simple, fast, and practical non-blocking and blocking
 1016 concurrent queue algorithms. In *Proceedings of the Fifteenth Annual ACM Symposium on*
 1017 *Principles of Distributed Computing*, PODC '96, page 267–275, New York, NY, USA, 1996.
 1018 Association for Computing Machinery. doi:10.1145/248052.248106.
- 1019 33 Egor Sergeevich Namakonov, Justus Fasse, Bart Jacobs, Lars Birkedal, and Amin Timany.
 1020 Lawyer: Modular Obligations-Based Liveness Reasoning in Higher-Order Impredicative Con-
 1021 current Separation Logic. In *Object-Oriented Programming, Systems, Languages & Applications*
 1022 *2026*, 2026.
- 1023 34 Christina Peterson, Victor Cook, and Damian Dechev. Practical progress verification of
 1024 descriptor-based non-blocking data structures. In *2019 IEEE 27th International Symposium on*
 1025 *Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*,
 1026 pages 83–93, 2019. doi:10.1109/MASCOTS.2019.00019.
- 1027 35 Simon Spies, Lennard Gäher, Daniel Gratzer, Joseph Tassarotti, Robbert Krebbers, Derek
 1028 Dreyer, and Lars Birkedal. Transfinite iris: resolving an existential dilemma of step-indexed
 1029 separation logic. In Stephen N. Freund and Eran Yahav, editors, *PLDI '21: 42nd ACM*
 1030 *SIGPLAN International Conference on Programming Language Design and Implementation*,
 1031 *Virtual Event, Canada, June 20-25, 2021*, pages 80–95. ACM, 2021. doi:10.1145/3453483.
 1032 3454031.
- 1033 36 David Swasey, Deepak Garg, and Derek Dreyer. Robust and compositional verification of
 1034 object capability patterns. *Proc. ACM Program. Lang.*, 1(OOPSLA), October 2017. doi:
 1035 10.1145/3133913.
- 1036 37 Amin Timany, Simon Oddershede Gregersen, Léo Stefanescu, Jonas Kastberg Hinrichsen, Léon
 1037 Gondelman, Abel Nieto, and Lars Birkedal. Trillium: Higher-order concurrent and distributed
 1038 separation logic for intensional refinement. *Proc. ACM Program. Lang.*, 8(POPL), jan 2024.
 1039 doi:10.1145/3632851.
- 1040 38 Shahar Timnat, Anastasia Braginsky, Alex Kogan, and Erez Petrank. Wait-free linked-lists.
 1041 *SIGPLAN Not.*, 47(8):309–310, February 2012. doi:10.1145/2370036.2145869.
- 1042 39 Viktor Vafeiadis. *Modular fine-grained concurrency verification*. PhD thesis, University of
 1043 Cambridge, 2008.
- 1044 40 Martin Vechev and Eran Yahav. Deriving linearizable fine-grained concurrent objects. *SIG-*
 1045 *PLAN Not.*, 43(6):125–135, June 2008. doi:10.1145/1379022.1375598.

1046 A Definitions used in the conditional adequacy theorem

1047 In Section 6, we used the notion of a progress resource. Below, we define it formally. In the
 1048 Rocq development, we provide a slightly more general version which accounts for threads
 1049 getting stuck and/or forking. Also, note that in Section 6 we treated progress resource as a
 1050 predicate with one argument, as its second argument was not relevant for the discussion.

1051 ► **Definition 14** (Progress resource). *Consider a labeled transition system $\mathcal{M}$. For it,*
 1052 *consider a trace interpretation and a trace invariant predicates $\text{TI}, \text{TrInv} : \text{FinTrace}(\lambda_{\text{ref}}^{\text{conc}}) \rightarrow$*
 1053 *$\text{FinTrace}(\mathcal{M}) \rightarrow i\text{Prop}$, where the latter is persistent. Then, consider a postcondition*
 1054 *predicate $\text{Post} : \text{Thread} \rightarrow \text{Val} \rightarrow i\text{Prop}$. Finally, consider a predicate on finite execution*
 1055 *traces $F : \text{FinTrace}(\lambda_{\text{ref}}^{\text{conc}}) \rightarrow \text{Prop}$.*

1056 *With that, we call any $\text{PR}_F : \text{FinTrace}(\lambda_{\text{ref}}^{\text{conc}}) \rightarrow \text{List}(\text{Val} \rightarrow i\text{Prop}) \rightarrow i\text{Prop}$ a progress*
 1057 *resource if it satisfies the two following laws:*

1058 $\forall \text{etr} : \text{FinTrace}(\lambda_{\text{ref}}^{\text{conc}}), \text{mtr} : \text{FinTrace}(\mathcal{M}), \Phi s : \text{List}(\text{Val} \rightarrow i\text{Prop}).$
 1059 $F(\text{etr}) \multimap \text{PR}_F(\text{etr}, \Phi s) \multimap \text{TI}(\text{etr}, \text{mtr}) \Rightarrow$ (prPosts)
 1060 $\text{postsOf}(\text{etr}.\text{last}.1) * \text{TI}(\text{etr}, \text{mtr}) * (\text{postsOf}(\text{etr}.\text{last}.1) \multimap \text{PR}_F(\text{etr}, \Phi s))$
 1061
 1062 $\forall \text{etr} : \text{FinTrace}(\lambda_{\text{ref}}^{\text{conc}}), \text{mtr} : \text{FinTrace}(\mathcal{M}), \Phi s : \text{List}(\text{Val} \rightarrow i\text{Prop}), c' : \text{Config}, \tau : \text{Thread}.$
 1063 $\text{etr}.\text{last} \xrightarrow{\tau} c' \multimap \text{TI}(\text{etr}, \text{mtr}) \multimap \text{TrInv}(\text{etr}) \multimap \text{PR}_F(\text{etr}, \Phi s) \multimap F(\text{etr} \vdash c')$
 1064 $\top \Rightarrow \emptyset \quad (\Rightarrow \emptyset \triangleright)^{|\text{etr}|+1} \quad \emptyset \Rightarrow \top$ (prTakeStep)
 1065 $\exists \delta : \text{State}(\mathcal{M}), \ell : \text{Label}(\mathcal{M}). \text{TI}(\text{etr} \vdash c', \text{mtr} \vdash \delta) * \text{TrInv}(\text{etr} \vdash c', \text{mtr} \vdash \delta) *$
 1066 $\text{PR}_F(\text{etr} \vdash c', \Phi s ++ \text{newPosts}(\text{etr}.\text{last}.1, c'.1))$

1067 Above, $tr \vdash x$ denotes extending a finite trace tr with a label y and state x .
 1068 The two definitions related to postconditions are formally defined in the Rocq development:

1069 ■ $\text{postsOf}(\mathcal{E} : \text{TPool})$ gives postconditions of threads of $\mathcal{E}$ that have terminated,
 1070 ■ $\text{newPosts}(\mathcal{E}_1, \mathcal{E}_2)$ adds postcondition predicates to threads forked when thread pool $\mathcal{E}_1$
 1071 has stepped into $\mathcal{E}_2$

1072 B Lawyer specifications of non-blocking properties

1073 In Section 4, we claimed that the specification $\text{NolnfExec}(\text{op})$ captures that all calls to op
 1074 must terminate. We, furthermore claimed that $\text{PresInv}(\text{op})$ implies that op does not rely on
 1075 any invariants that involve Lawyer resources. Thus, according to Liang and Feng [30], these
 1076 conditions together imply that op is wait-free as it must never block itself, nor can it delay
 1077 other threads. Below we will explain how these specifications prohibit blocking and delaying,
 1078 and discuss the changes needed in our approach in order to support other non-blocking
 1079 properties.

1080 To understand why $\text{NolnfExec}(\text{op})$ prohibits blocking, we need to explain what this
 1081 specification *does not* include. Remember that it is defined as follows, for some fixed $\mathbf{F} : \mathbb{N}$:

$$1082 \quad \forall \tau : \text{Thread}, \pi : \text{Phase}, a : \text{Val}. \{\mathbf{F} \cdot \text{bar}_\circ \pi * \text{ph}_\circ \tau \pi\} \text{op } a \{ _ . \text{ph}_\circ \tau \pi \}^\tau$$

1083 Neither its precondition- nor postcondition mention any obligation resource $\text{obls } \tau \mathcal{O}$ of
 1084 Lawyer. This resource indicates that the thread τ holds a set $\mathcal{O}$ of *obligations* that it must
 1085 fulfill, *e.g.*, setting a shared flag that other threads can wait for. In Lawyer, a thread having
 1086 an obligation allows the other threads to busy-wait for this obligation to be fulfilled. In
 1087 practice, waiting of another thread's obligation means that the waiting thread can *generate*
 1088 fuel to be spent during its busy-waiting. To prevent circular waiting, a thread can only
 1089 generate fuel by presenting its own set of obligations, and crucially proving that all their
 1090 obligations are “ordered after” the obligation (of the other thread) that it waits for.⁹ Thus,
 1091 $\text{NolnfExec}(\text{op})$ not including obligations resources means that it is non-blocking, because it
 1092 cannot generate fuel by waiting for other threads' obligations, and thus it can only spend
 1093 the fuel that is explicitly provided to it in $\text{NolnfExec}(\text{op})$.

⁹ For more details of what this “ordered after” relation is see the Lawyer paper. We only emphasize here that this order is well-founded.

1094 A delay in a lock-free algorithm is a situation when the successful completion of one
 1095 operation makes another operation running concurrently with it to fail and have to retry.
 1096 An example of it can be seen in Figure 1b where a successful **CAS** in one thread can cause
 1097 another thread to fail its own **CAS** which forces it attempt to increment again by making
 1098 a recursive call. To prohibit delays, our approach disables the fuel sharing mechanism of
 1099 Lawyer. This mechanism, introduced by [20], requires a thread that delays others to provide
 1100 them with extra fuel: since the delayed thread will retry its operation, it will need more fuel.
 1101 In Lawyer (see [33, Delaying example]), the fuel is shared via an invariant that is accessible
 1102 by both the delaying thread, and the delayed thread. To disable fuel sharing, we simply
 1103 require that invariants of the wait-free operation do not contain any Lawyer resources.¹⁰

1104 Besides wait-freedom, there are two other non-blocking properties: obstruction-freedom
 1105 [17] and lock-freedom. The former only requires an operation to progress if it is delayed by
 1106 other operations a finite number of times. When this condition is violated, an obstruction-
 1107 free operation might block. Thus, we believe that adapting our approach to verifying
 1108 obstruction-freedom could be achieved by adding obligation resources to the specification.
 1109 For lock-freedom, one could follow the approach of Lawyer and enable fuel sharing by
 1110 allowing fuel sharing, *i.e.* by lifting the aforementioned restriction on invariants. However,
 1111 the soundness of fuel sharing in Lawyer is relies on the “phases” mechanism [20], which only
 1112 allows to share fuel among the threads that have the same ancestor in the fork tree. The
 1113 definition of lock-freedom does not place any restrictions on threads’ ancestry, and therefore
 1114 this restriction too must be lifted from the fuel sharing mechanism.

¹⁰ More formally, we require that the invariant can be defined in non-Lawyer instantiations of Trillium.